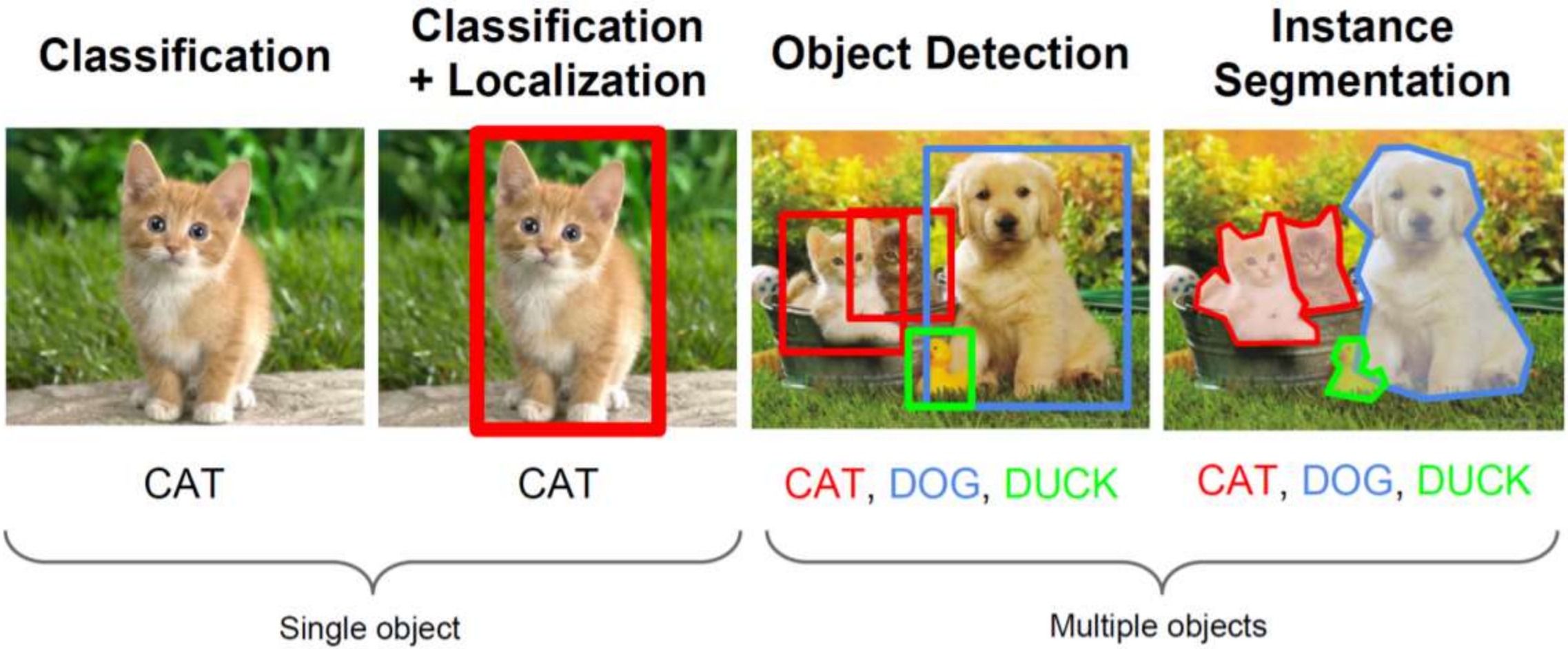
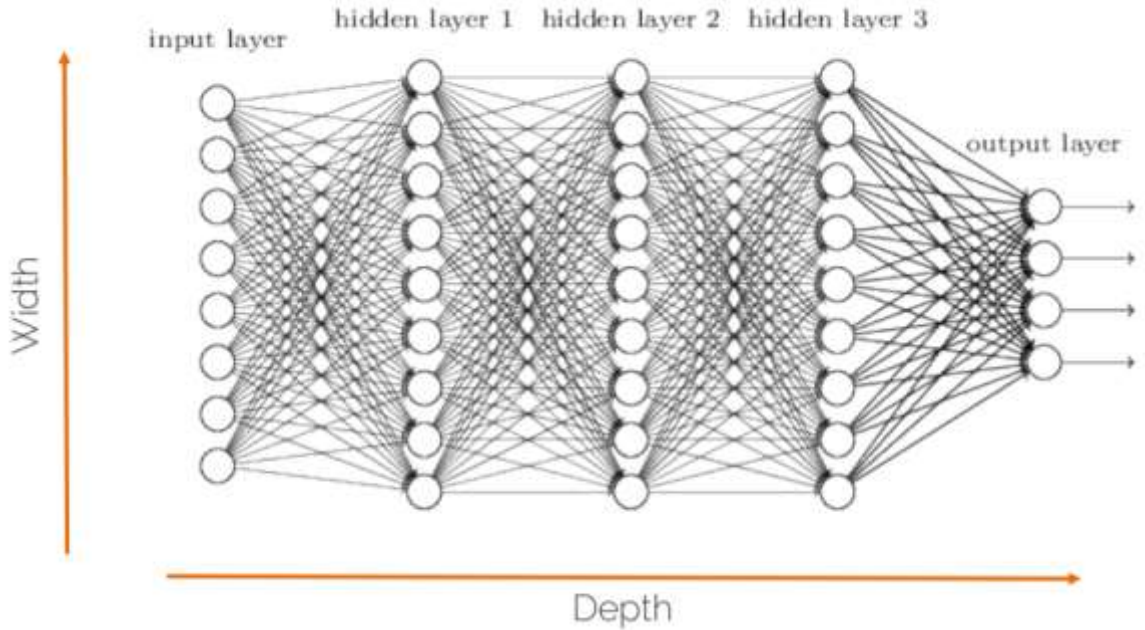
Convolutional Neural Networks

Using CNNs in Computer Vision



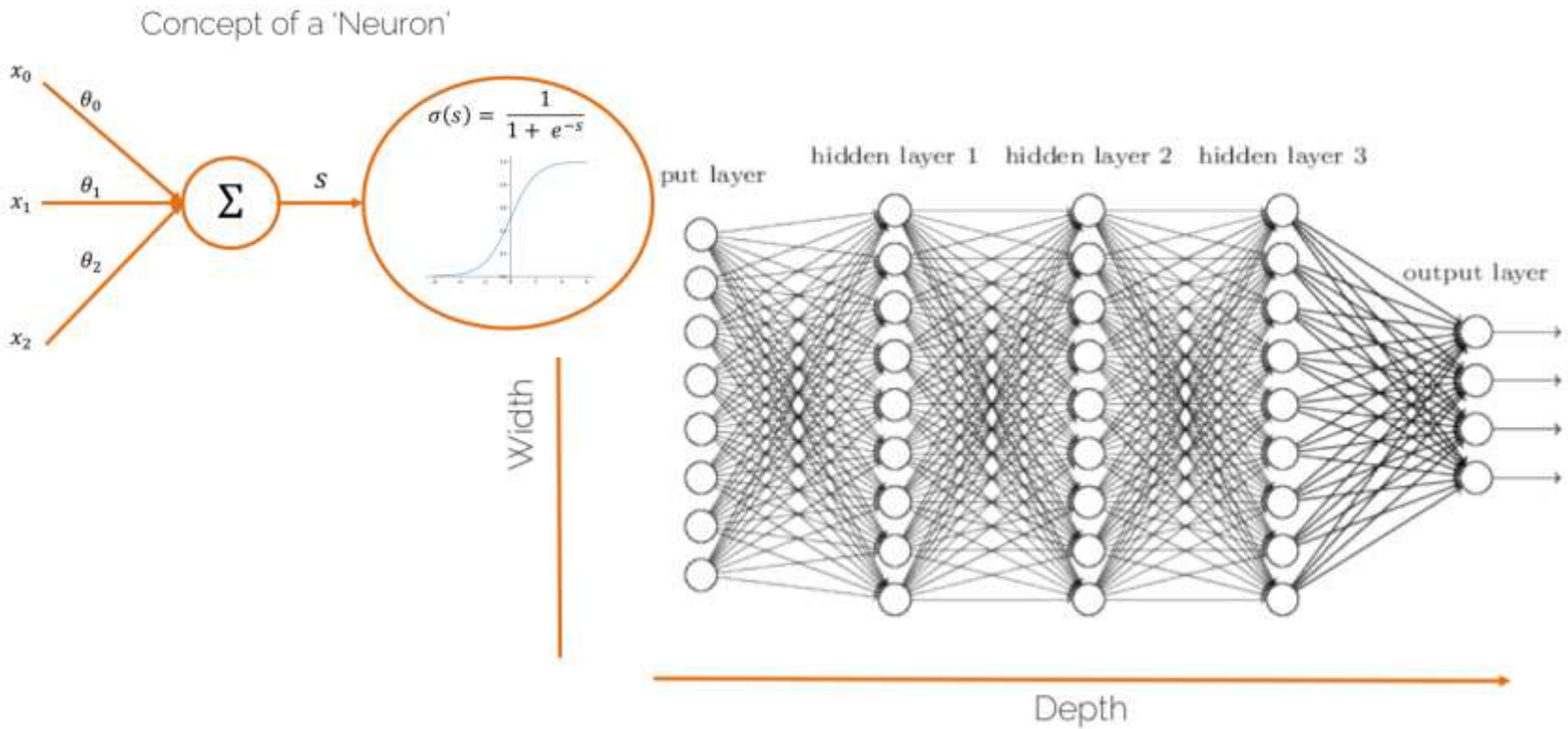
Source: Fei-Fei Li, Ranjay Krishna, Danfei Xu, CS231n: Convolutional Neural Networks for Visual Recognition

What do we know so far?



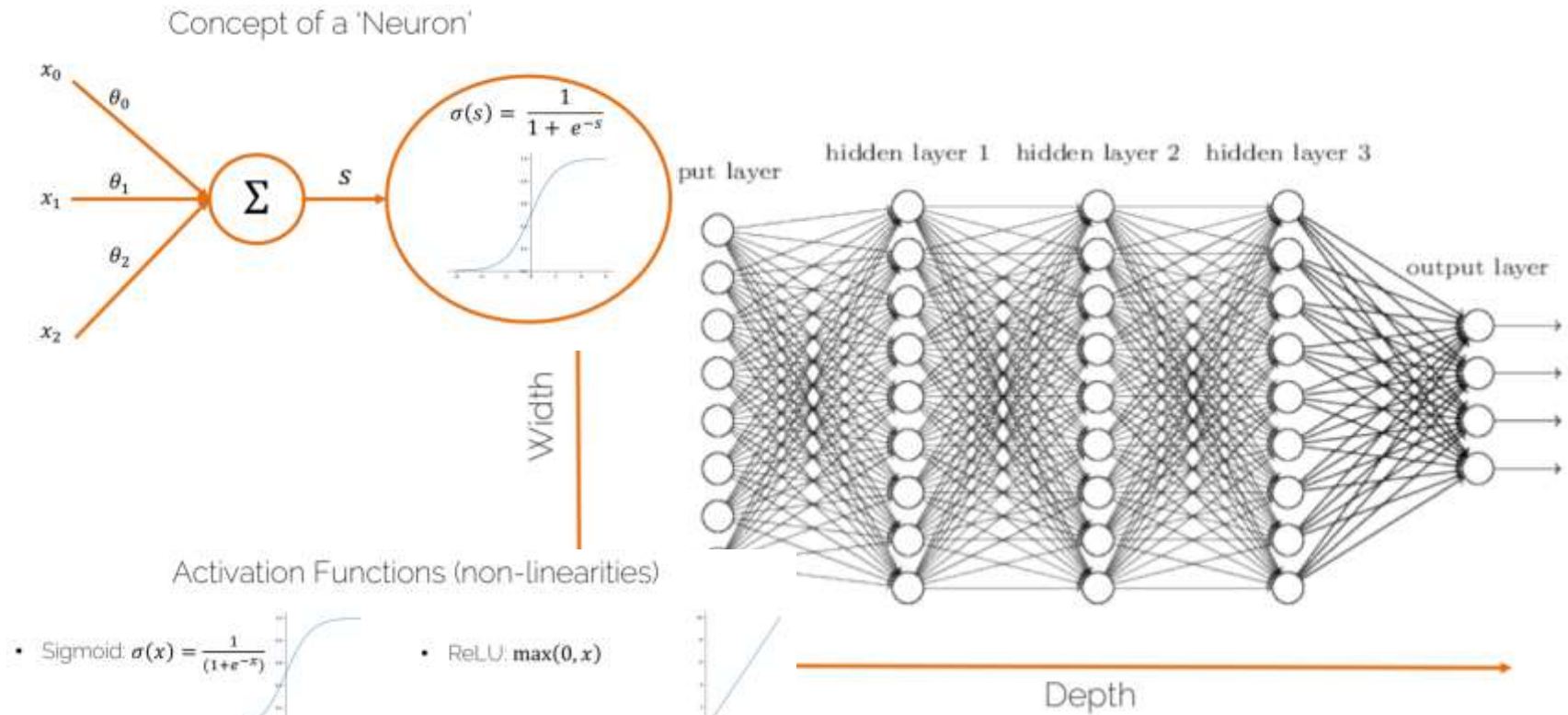
Source: Niessner and Leal-Taixé, Introduction to Deep Learning

What do we know so far?



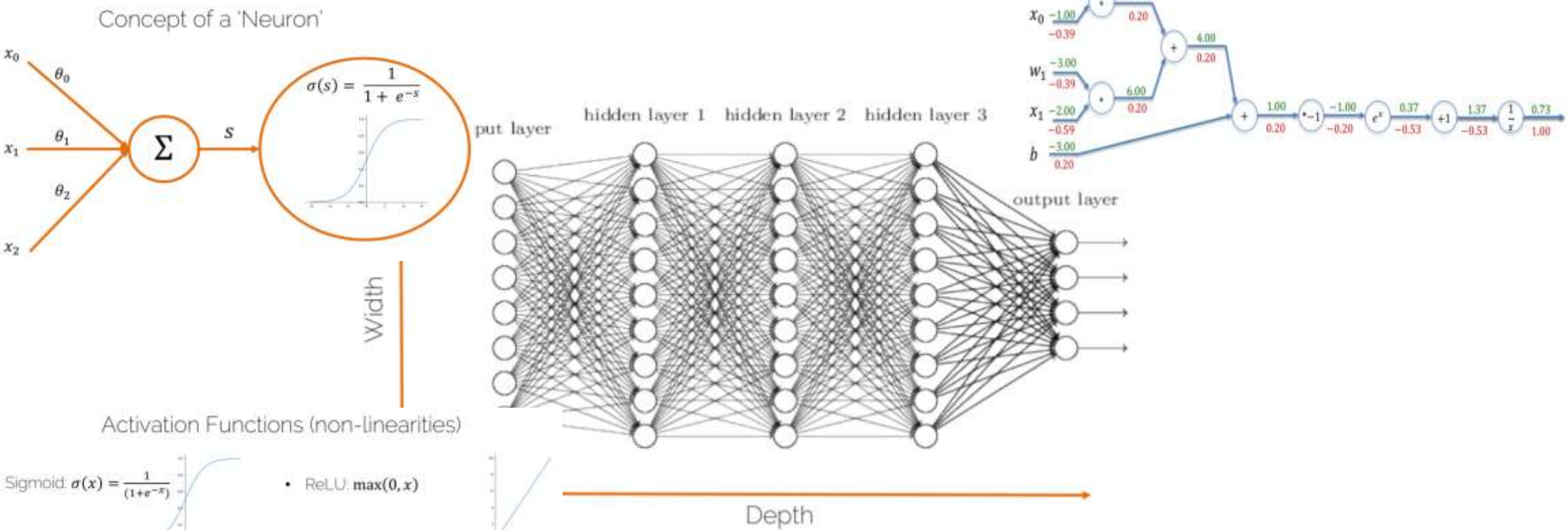
Source: Niessner and Leal-Taixé, Introduction to Deep Learning

What do we know so far?



Source: Niessner and Leal-Taixé, Introduction to Deep Learning

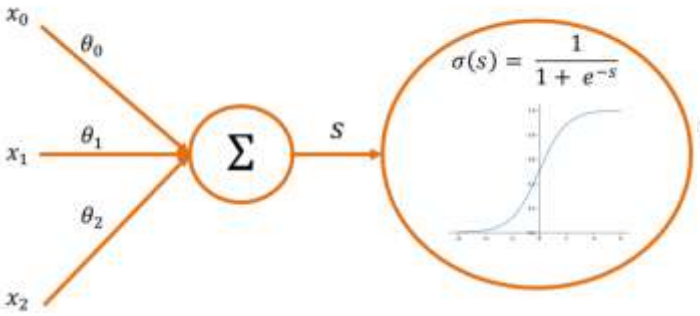
What do we know so far?



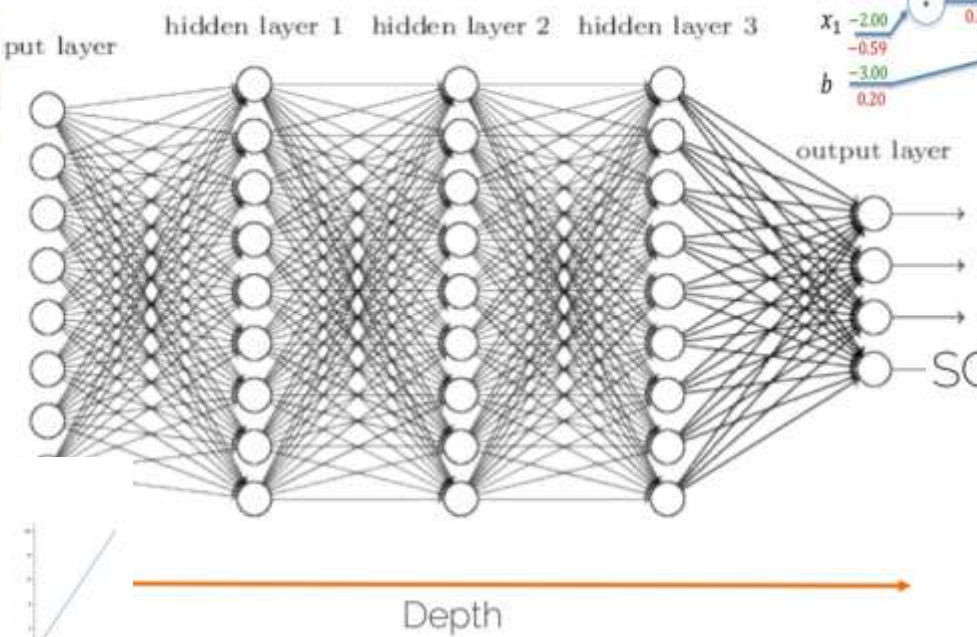
Source: Niessner and Leal-Taixé, Introduction to Deep Learning

What do we know so far?

Concept of a 'Neuron'



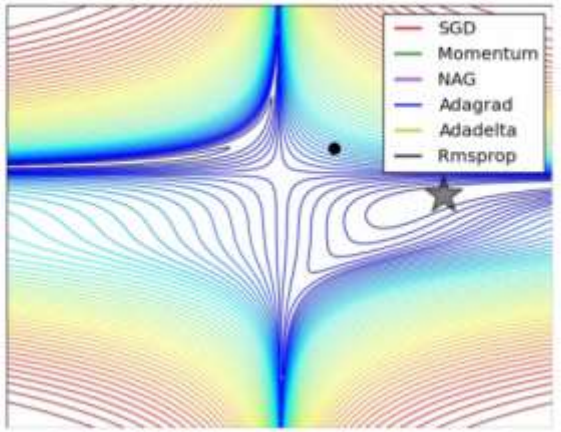
Backpropagation



Activation Functions (non-linearities)

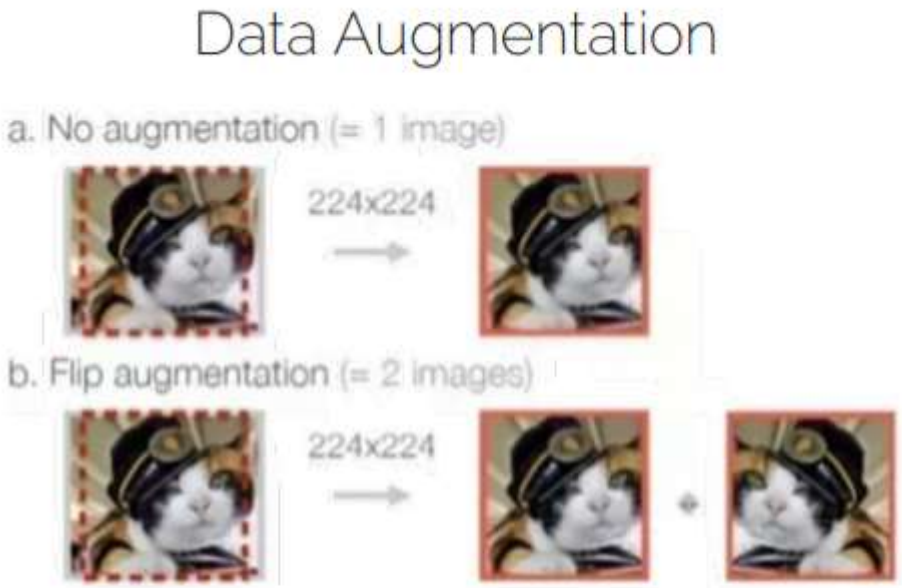
- Sigmoid: $\sigma(x) = \frac{1}{(1+e^{-x})}$
- ReLU: $\max(0, x)$
- TanH: $\tanh(x)$
- Leaky ReLU: $\max(0.1x, x)$

SGD Variations (Momentum, etc.)



Source: Niessner and Leal-Taixé, Introduction to Deep Learning

What do we know so far?



Source: Niessner and Leal-Taixé, Introduction to Deep Learning

What do we know so far?

Data Augmentation

a. No augmentation (= 1 image)

b. Flip augmentation (= 2 images)

Weight Regularization

e.g., L^2-reg: $R^2(\boldsymbol{W}) = \sum_{i=1}^{N} w_i^2$

Source: Niessner and Leal-Taixé, Introduction to Deep Learning

What do we know so far?

Data Augmentation

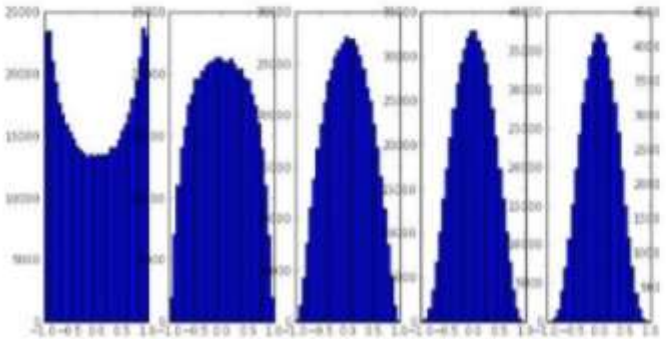
a. No augmentation (= 1 image)

b. Flip augmentation (= 2 images)

Weight Regularization

e.g., L^2-reg: $R^2(\boldsymbol{W}) = \sum_{i=1}^{N} w_i^2$

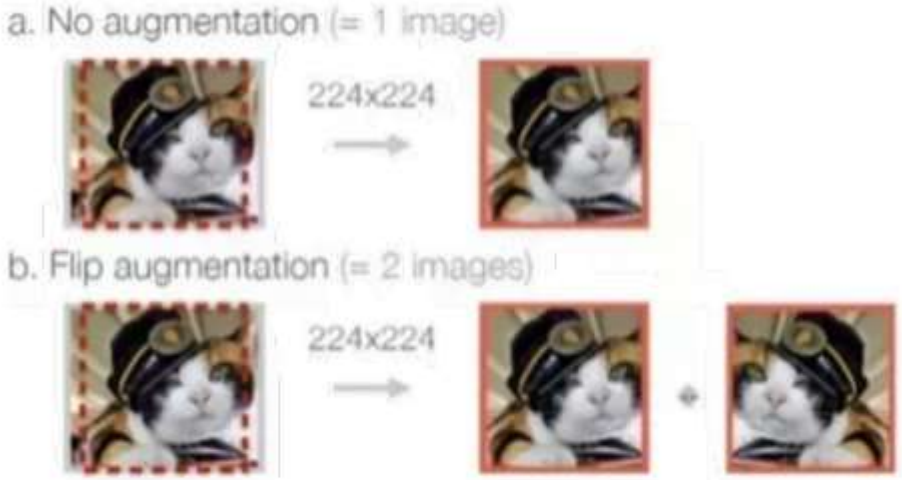
Weight Initialization (e.g., Xavier/2)



Source: Niessner and Leal-Taixé, Introduction to Deep Learning

What do we know so far?

Data Augmentation



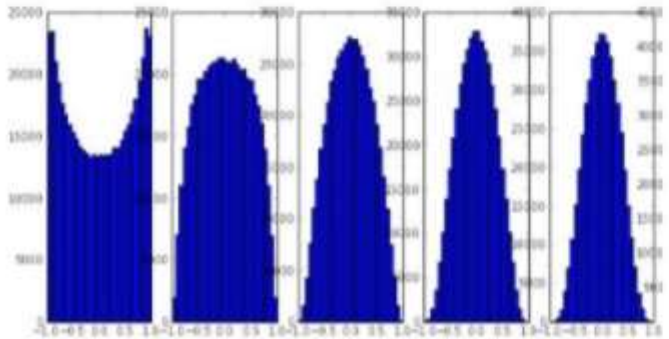
Batch-Norm

$$\hat{\boldsymbol{x}}^{(k)} = \frac{\boldsymbol{x}^{(k)} - E[\boldsymbol{x}^{(k)}]}{\sqrt{Var[\boldsymbol{x}^{(k)}]}}$$

Weight Regularization

e.g., L^2-reg: $R^2(\boldsymbol{W}) = \sum_{i=1}^{N} w_i^2$

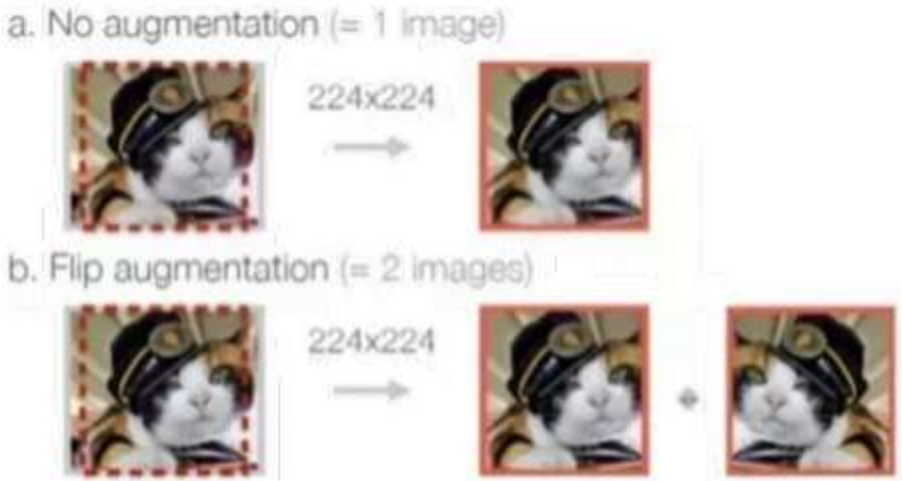
Weight Initialization (e.g., Xavier/2)



Source: Niessner and Leal-Taixé, Introduction to Deep Learning

What do we know so far?

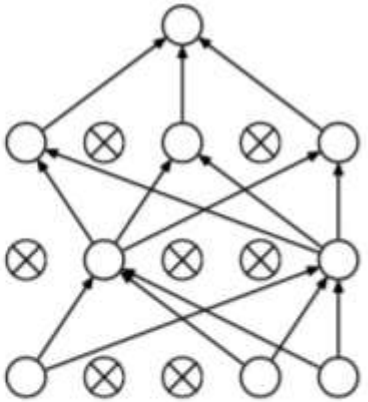
Data Augmentation



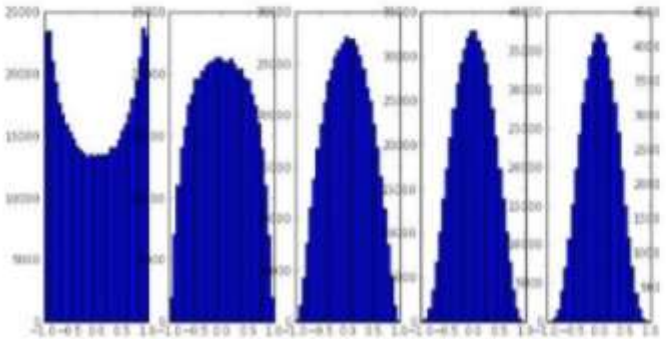
Batch-Norm

$$\hat{\boldsymbol{x}}^{(k)} = \frac{\boldsymbol{x}^{(k)} - E[\boldsymbol{x}^{(k)}]}{\sqrt{Var[\boldsymbol{x}^{(k)}]}}$$

Dropout


(b) After applying dropout.

Weight Initialization (e.g., Xavier/2)

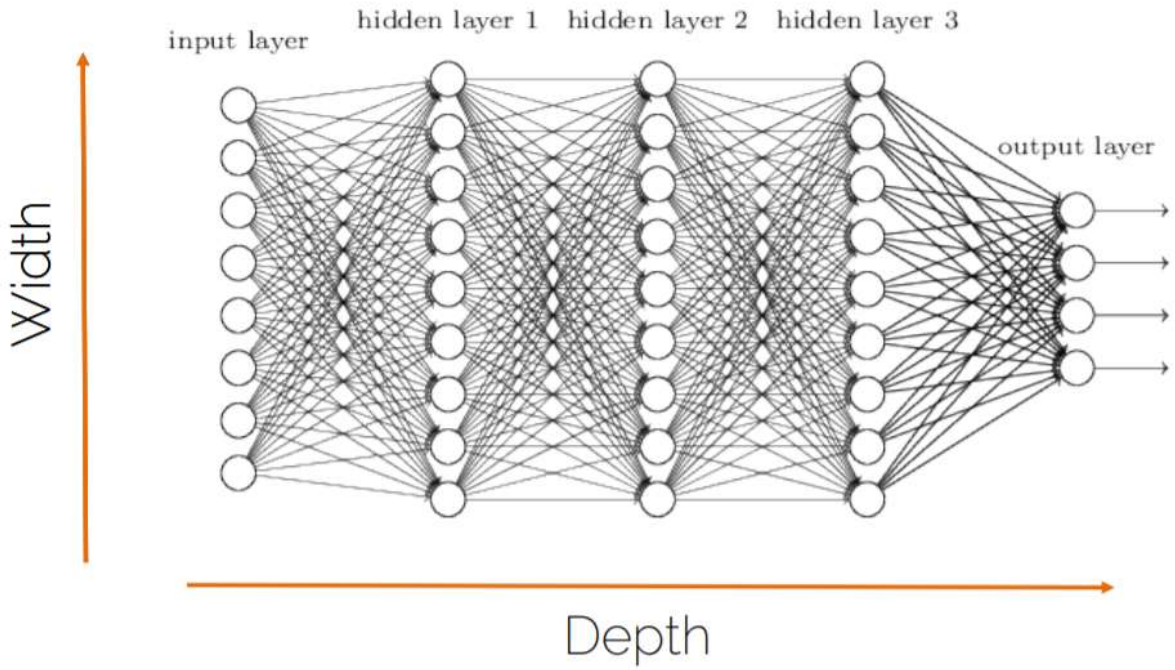


Weight Regularization

e.g., L^2-reg: $R^2(\boldsymbol{W}) = \sum_{i=1}^{N} w_i^2$

Source: Niessner and Leal-Taixé, Introduction to Deep Learning

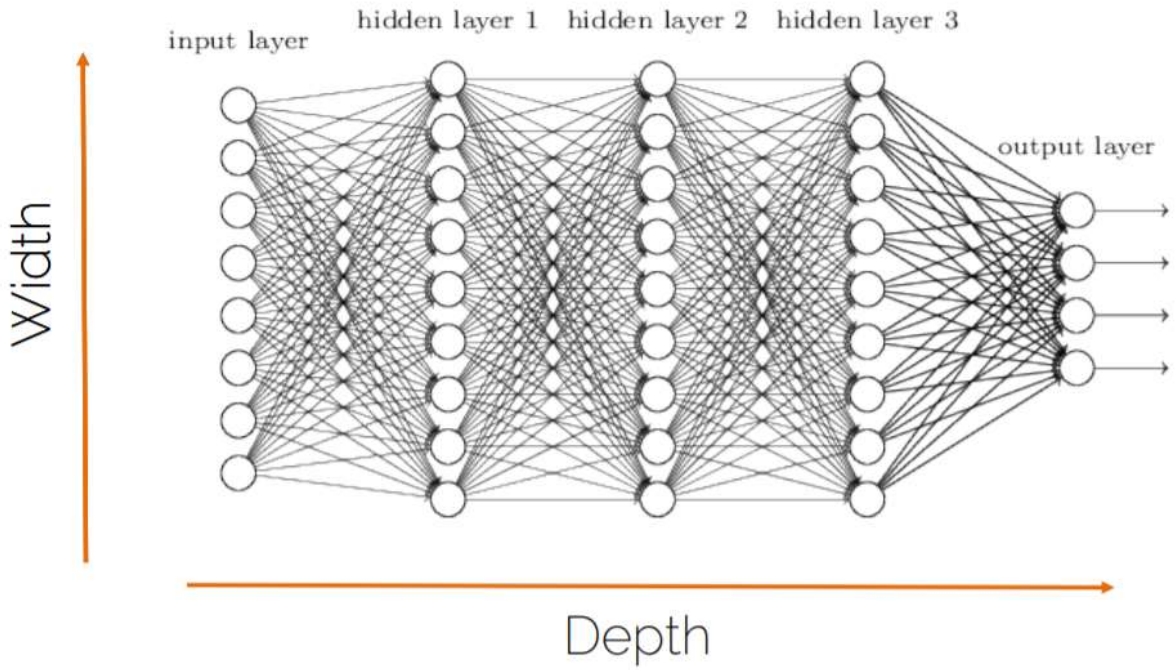
Why not simply more Layers?



Source: Niessner and Leal-Taixé, Introduction to Deep Learning

Why not simply more Layers?

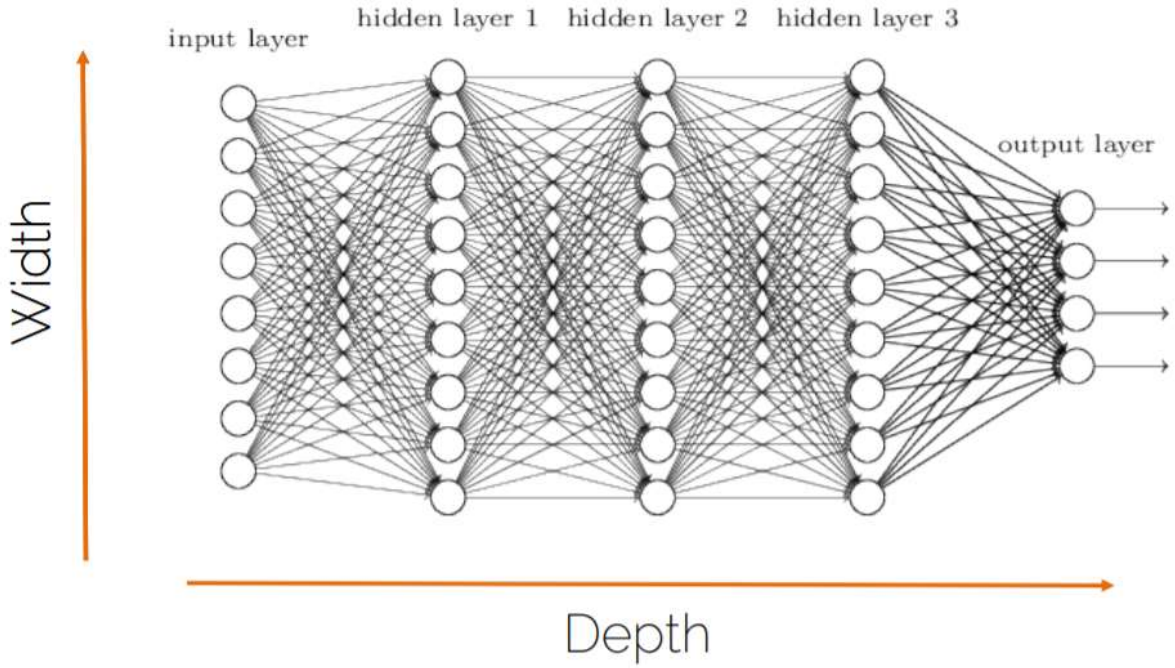
- We cannot make networks arbitrarily complex



Source: Niessner and Leal-Taixé, Introduction to Deep Learning

Why not simply more Layers?

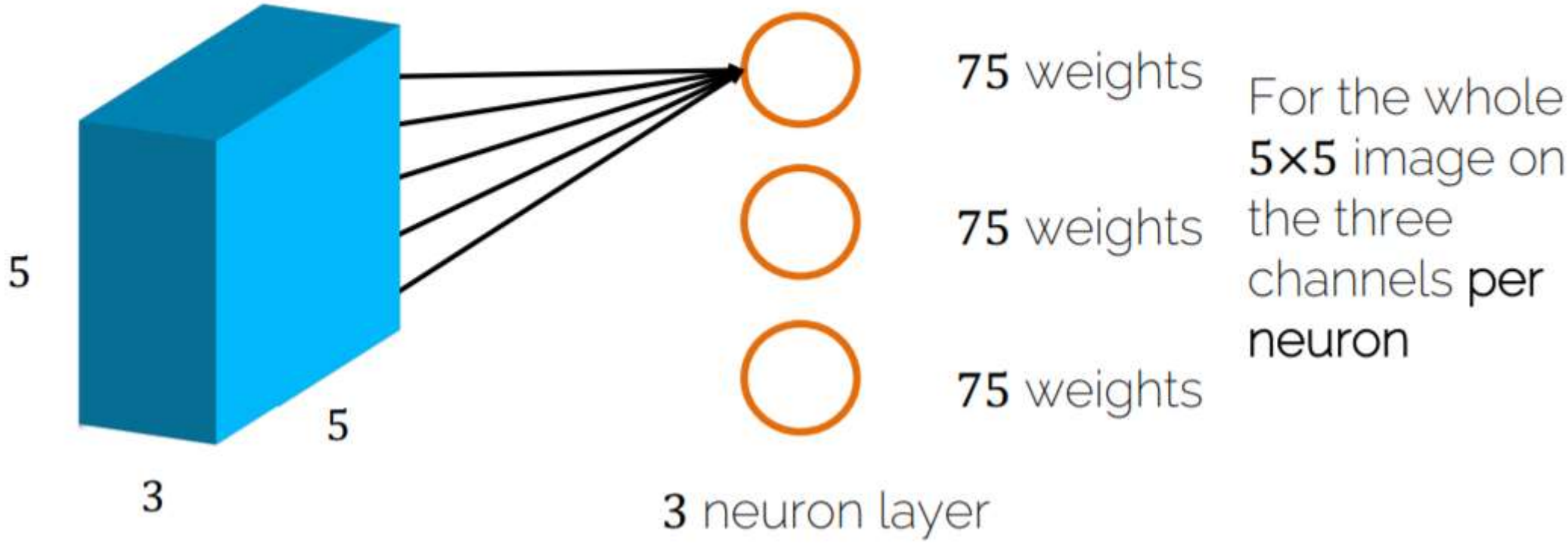
- We cannot make networks arbitrarily complex
- Why not just go deeper and get better?
  - No structure!!
  - It is just brute force!
  - Optimization becomes hard
  - Performance plateaus / drops!



Source: Niessner and Leal-Taixé, Introduction to Deep Learning

Problems using FC Layers on Images

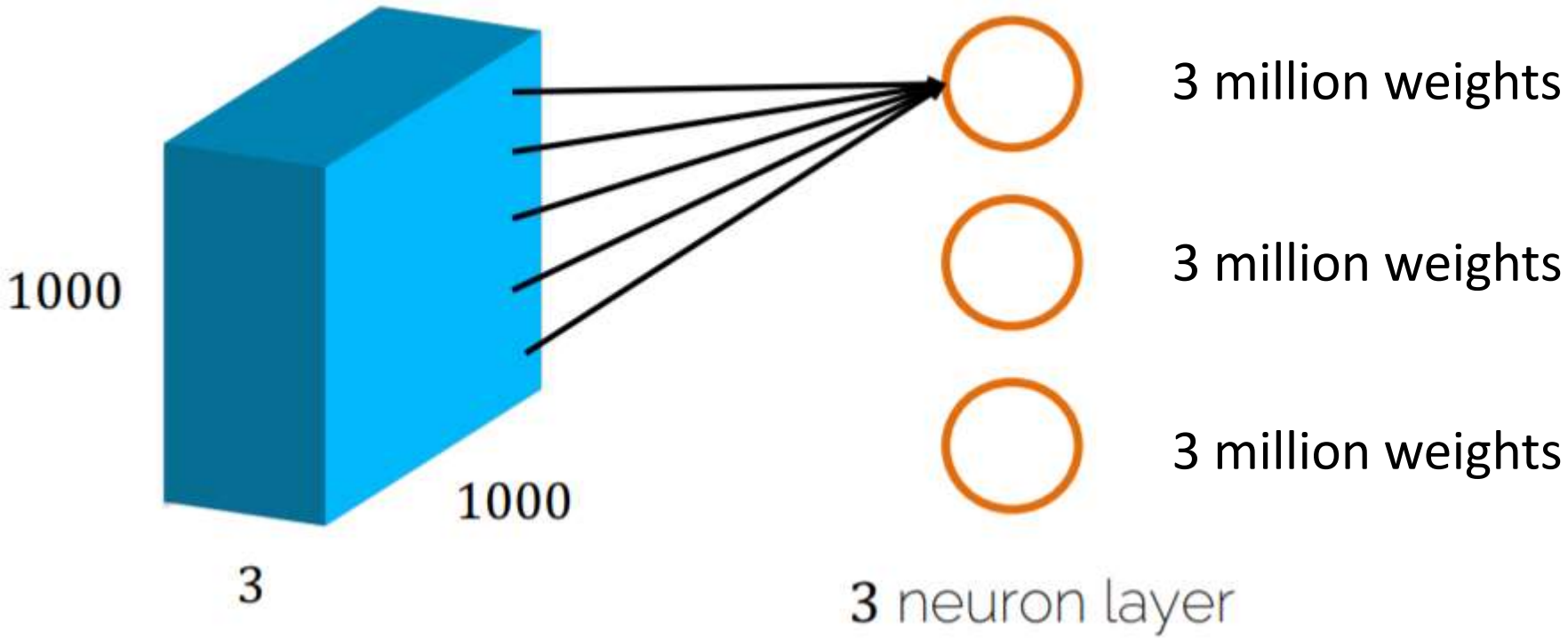
How to process a tiny image with FC layers



Source: Niessner and Leal-Taixé, Introduction to Deep Learning

Problems using FC Layers on Images

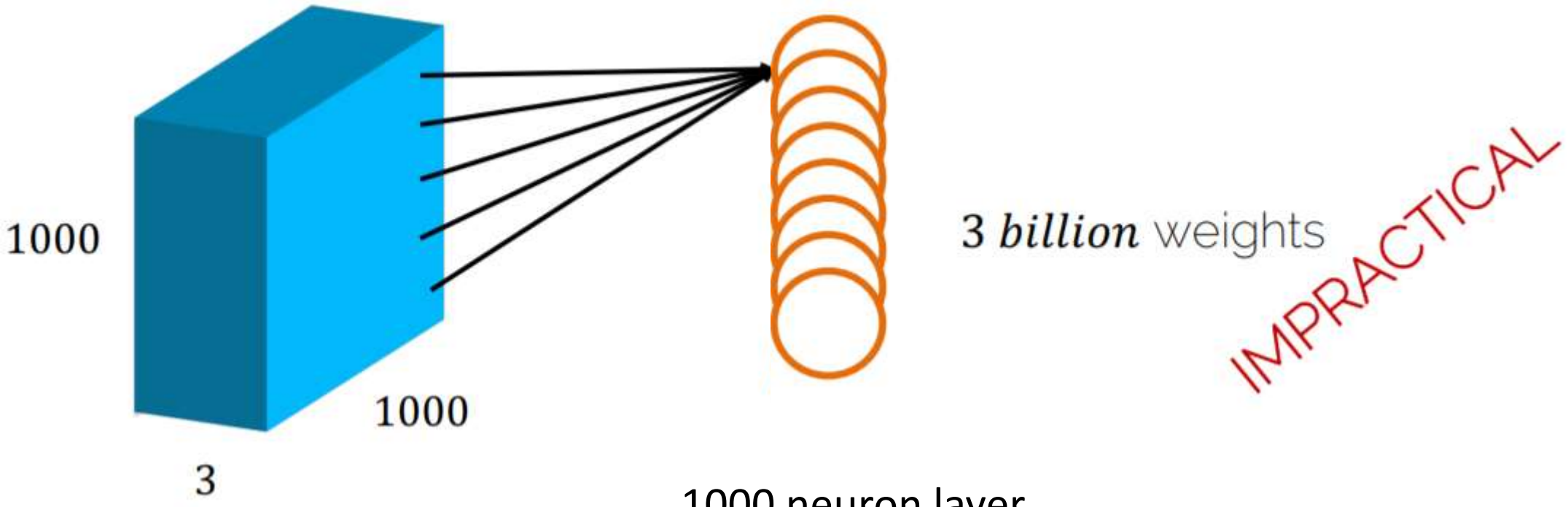
How to process a normal image with FC layers



Source: Niessner and Leal-Taixé, Introduction to Deep Learning

Problems using FC Layers on Images

How to process a normal image with FC layers



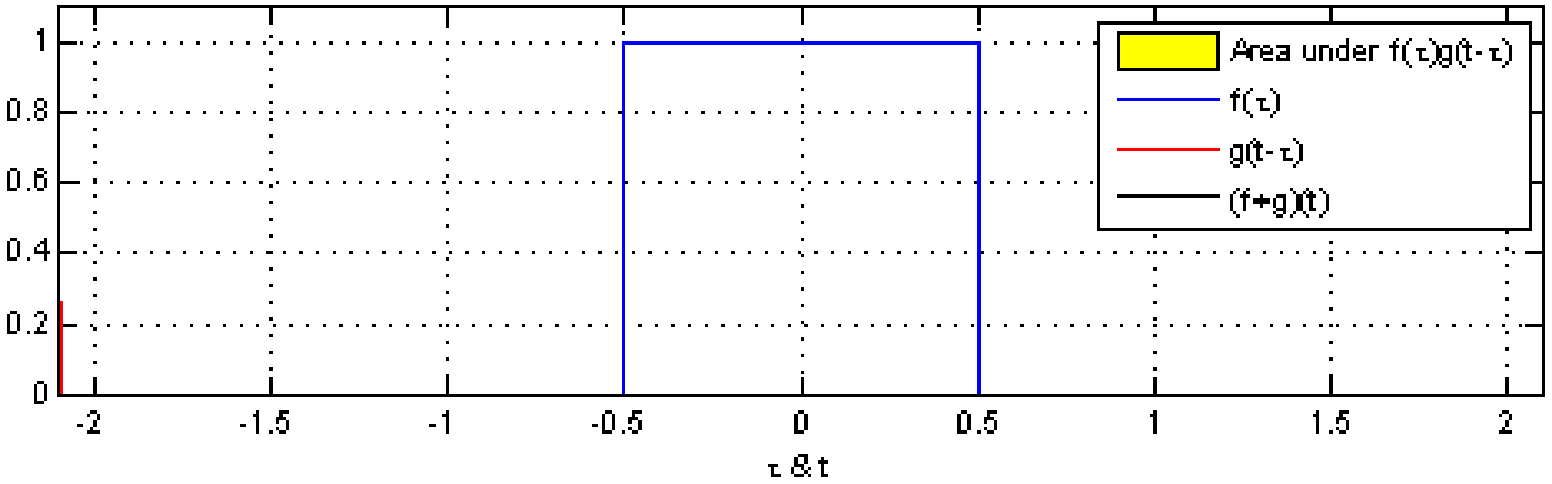
1000 neuron layer

Source: Niessner and Leal-Taixé, Introduction to Deep Learning

Better Way than FC ?

- We want to restrict the degrees of freedom
  - We want a layer with structure
  - Weight sharing -> using the same weights for different parts of the image

What are Convolutions?

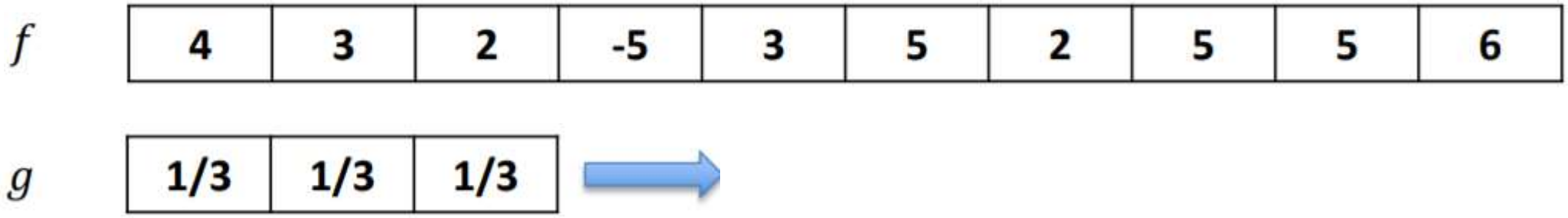


Convolution of two block function

Source: https://en.wikipedia.org/wiki/Convolution

What are Convolutions?

Discrete case: box filter

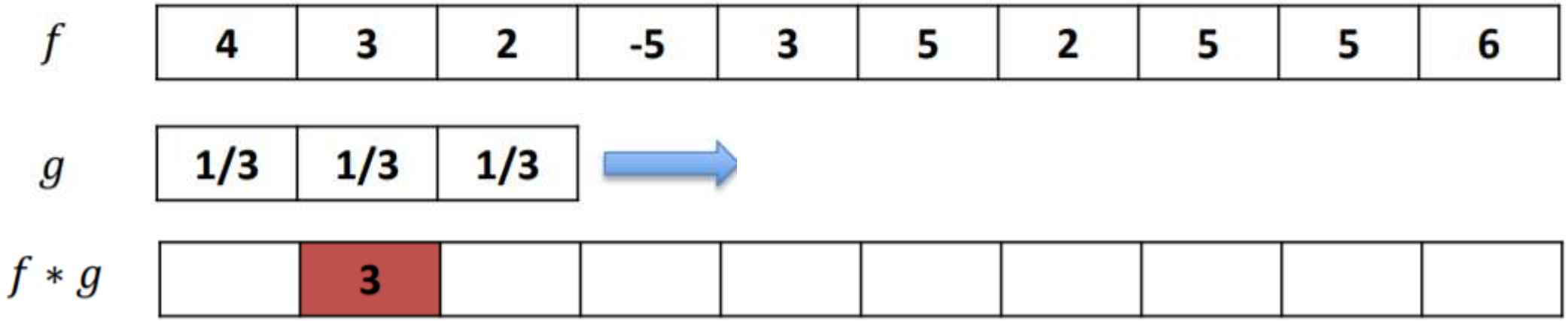


‘Slide’ filter kernel from left to right; at each position, compute a single value in the output data.

Source: Niessner and Leal-Taixé, Introduction to Deep Learning

What are Convolutions?

Discrete case: box filter

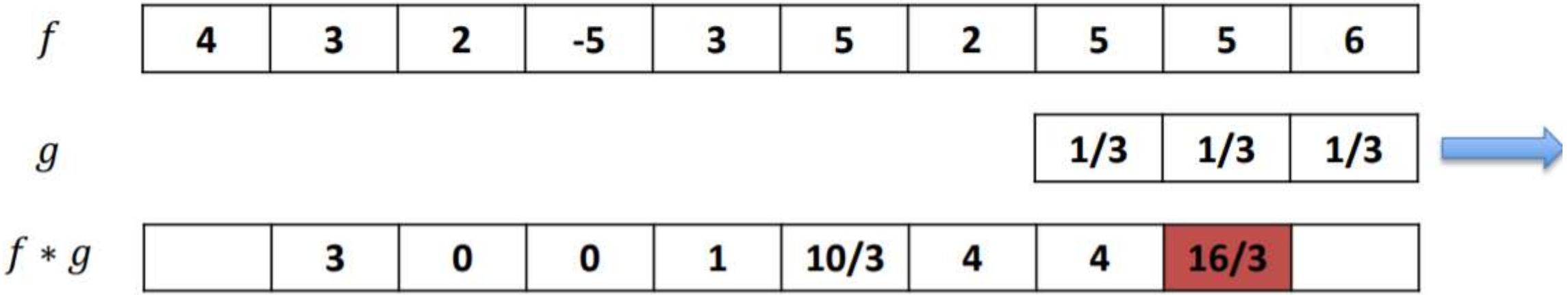


‘Slide’ filter kernel from left to right; at each position, compute a single value in the output data.

Source: Niessner and Leal-Taixé, Introduction to Deep Learning

What are Convolutions?

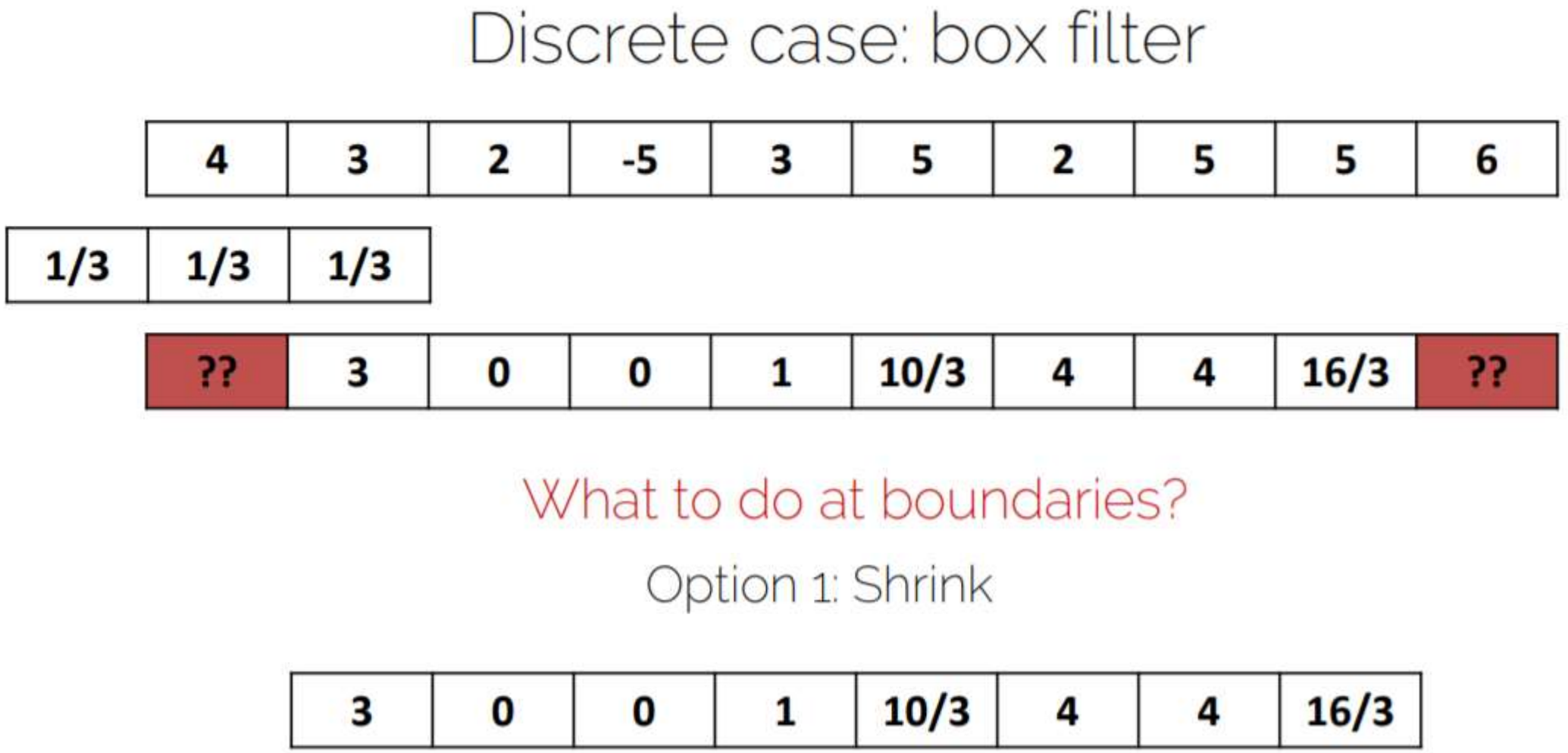
Discrete case: box filter



‘Slide’ filter kernel from left to right; at each position, compute a single value in the output data.

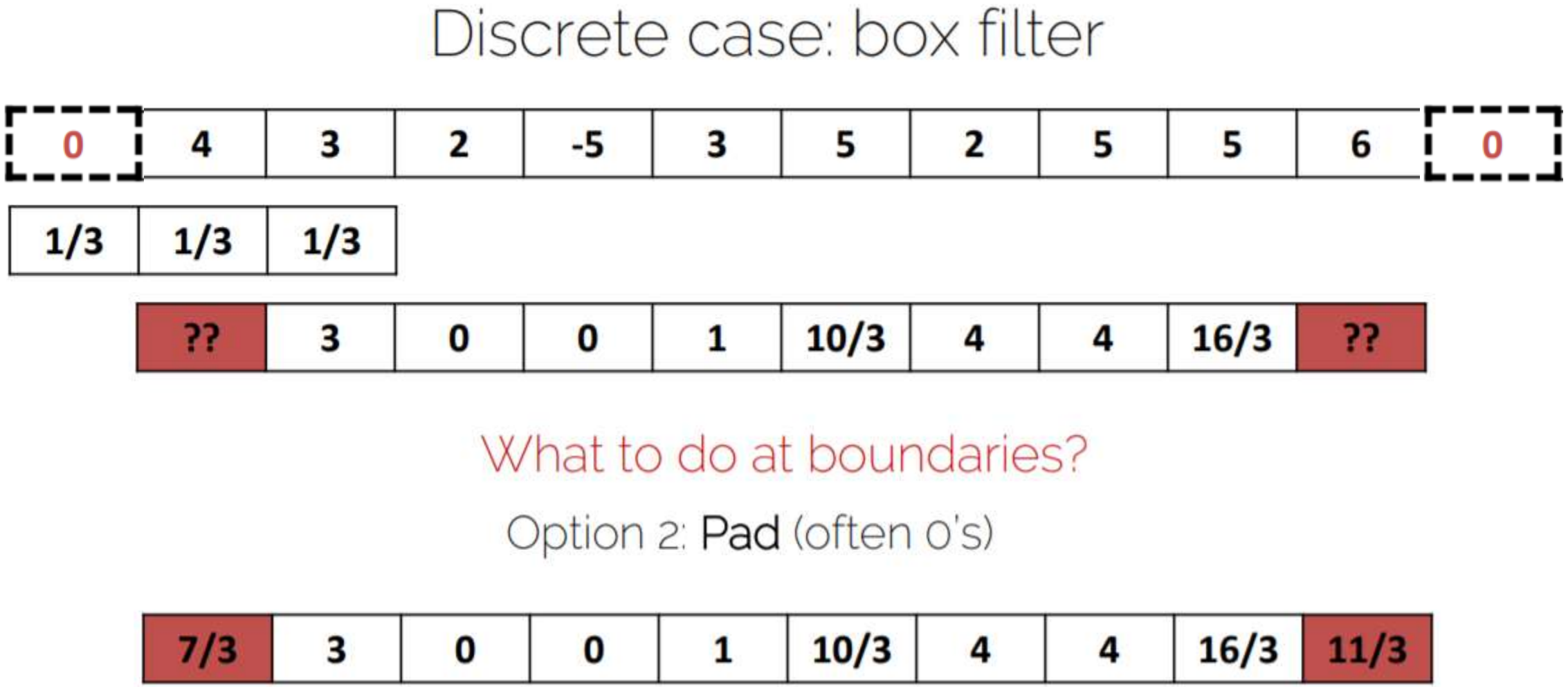
Source: Niessner and Leal-Taixé, Introduction to Deep Learning

What are Convolutions?



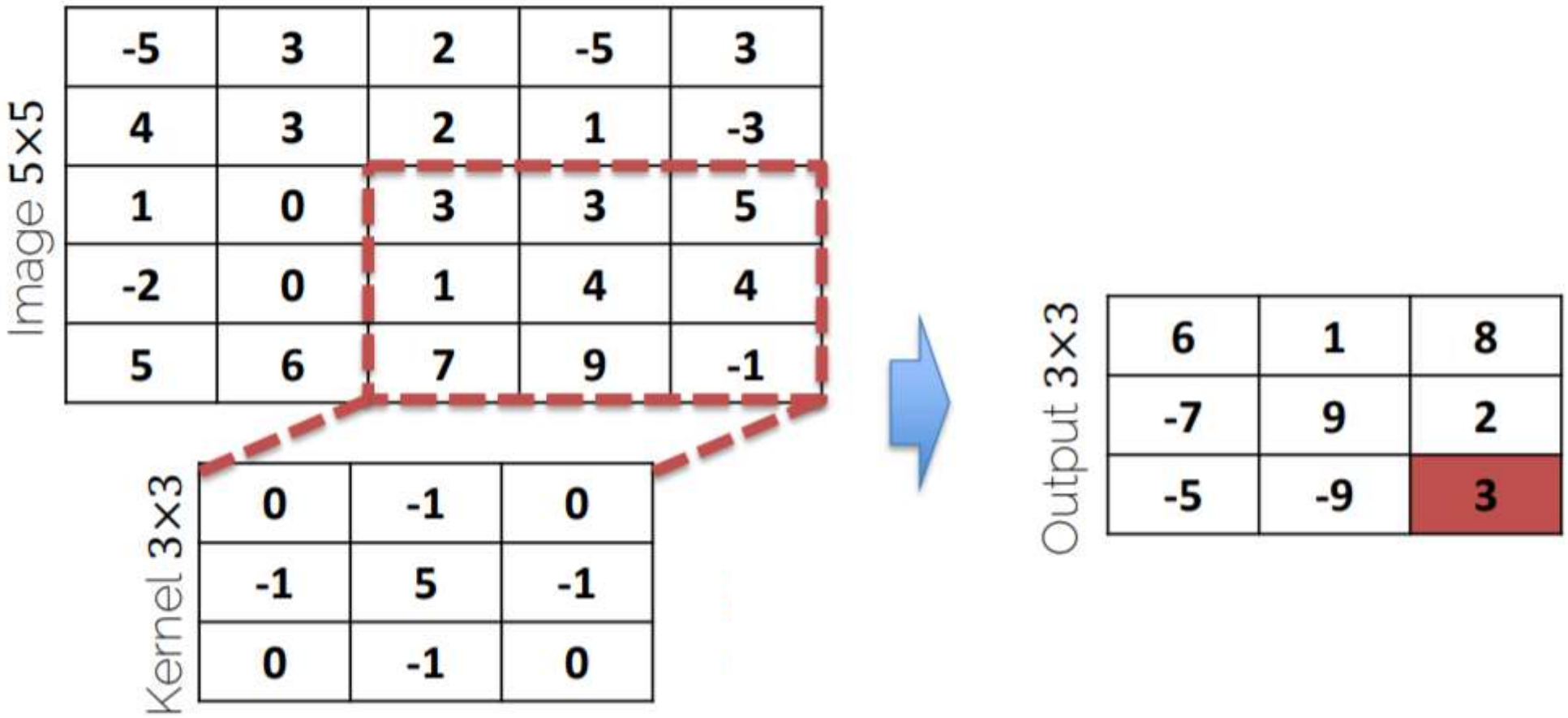
Source: Niessner and Leal-Taixé, Introduction to Deep Learning

What are Convolutions?



Source: Niessner and Leal-Taixé, Introduction to Deep Learning

Convolutions on Images



Source: Niessner and Leal-Taixé, Introduction to Deep Learning

Image Filters

- Each kernel gives us a different image filter

Input

Edge detection

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

Box mean

$$\frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

LET'S LEARN THESE FILTERS!

Sharpen

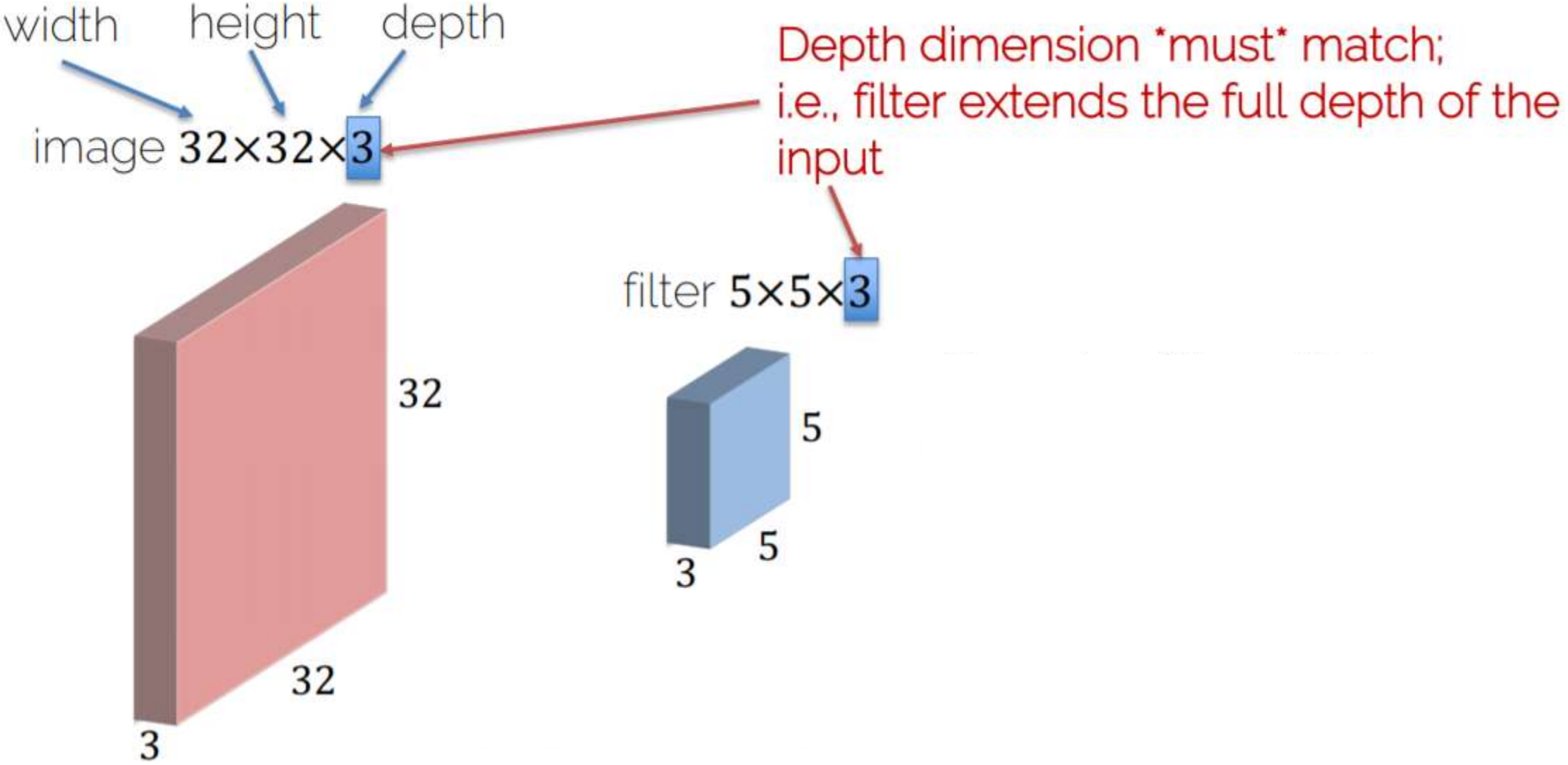
$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Gaussian blur

$$\frac{1}{16}\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

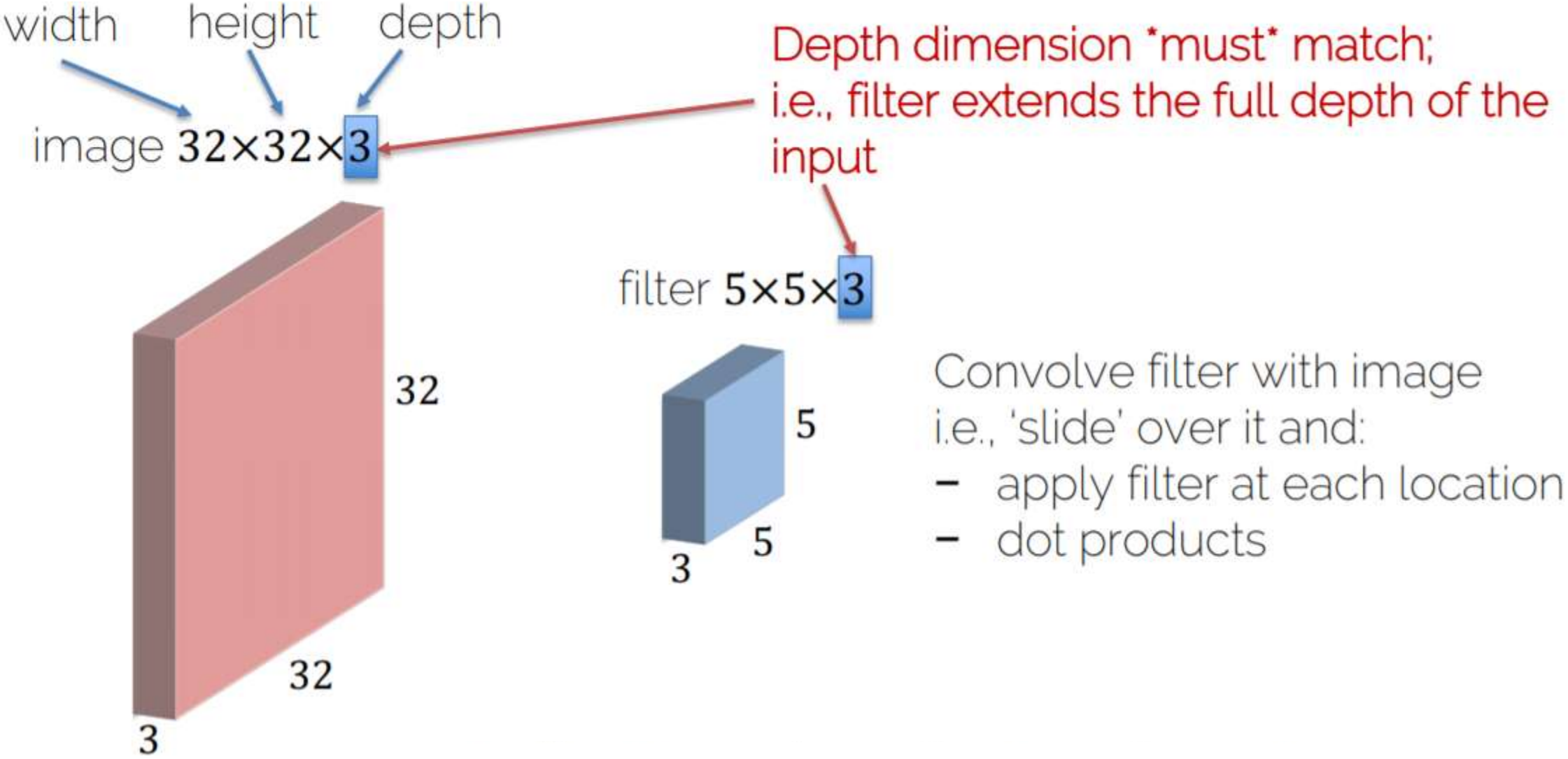
Source: Niessner and Leal-Taixé, Introduction to Deep Learning

Convolutions on RGB Images



Source: Niessner and Leal-Taixé, Introduction to Deep Learning

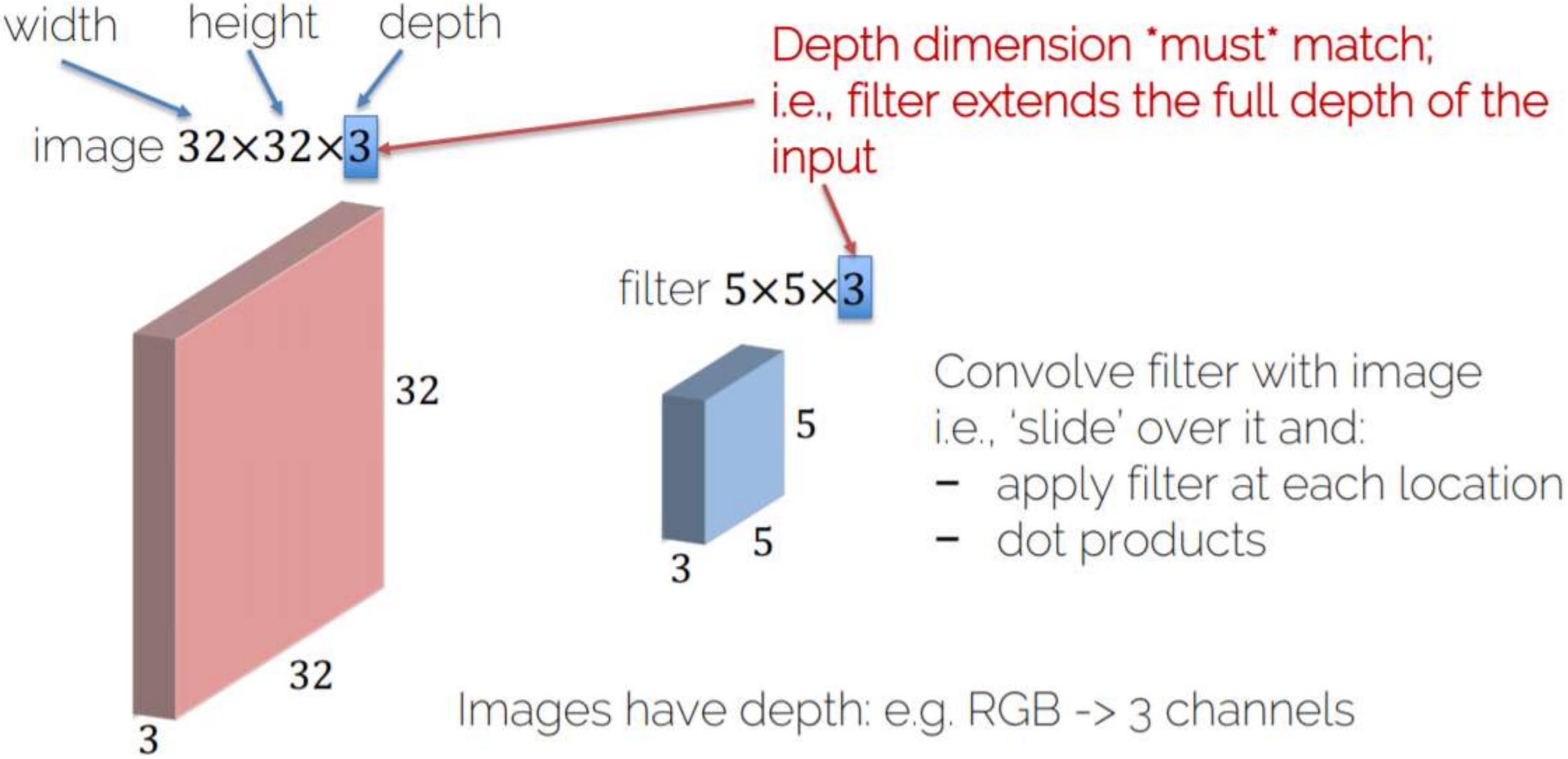
Convolutions on RGB Images



Convolve filter with image
i.e., 'slide' over it and:
- apply filter at each location
- dot products

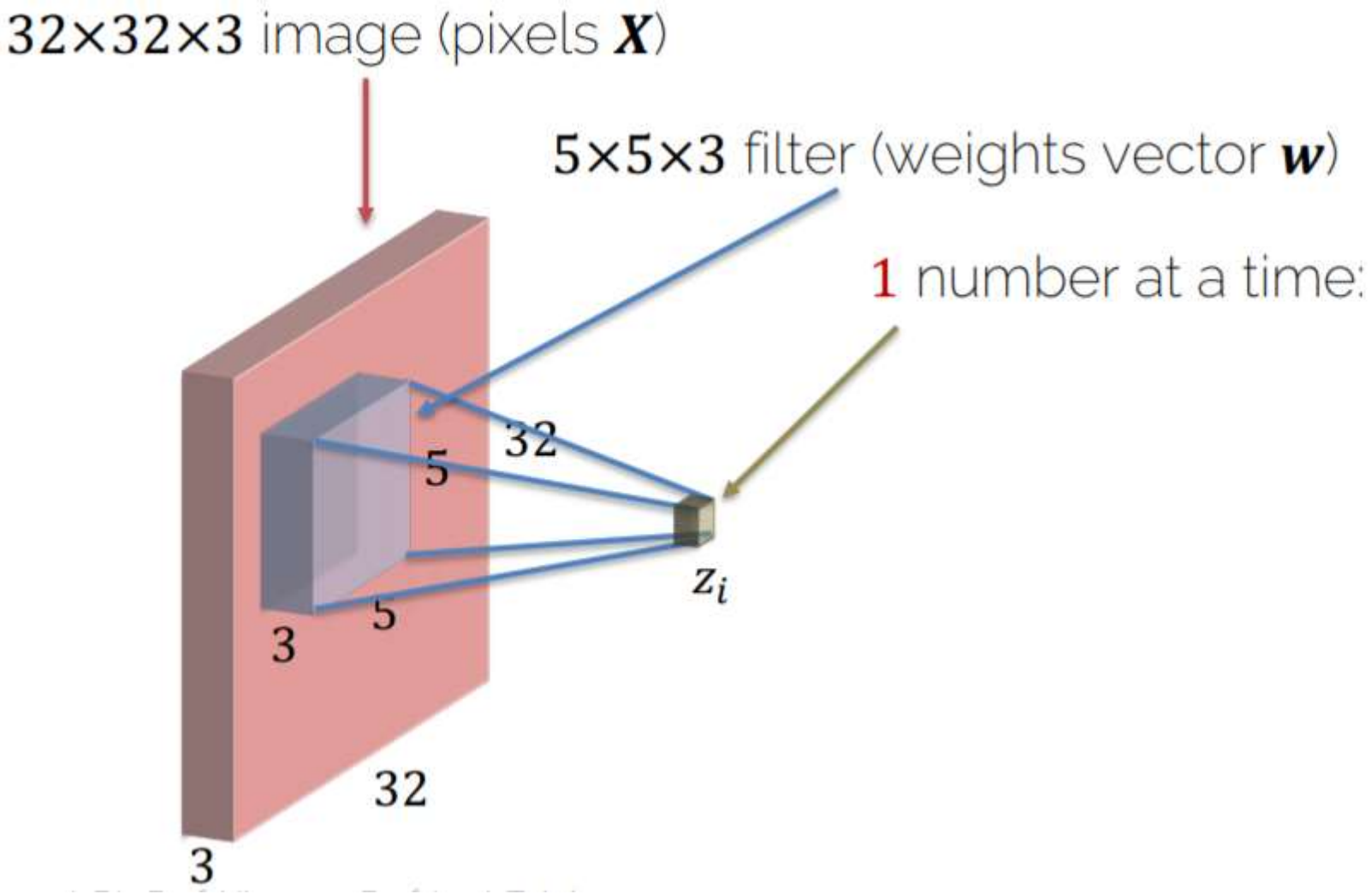
Source: Niessner and Leal-Taixé, Introduction to Deep Learning

Convolutions on RGB Images



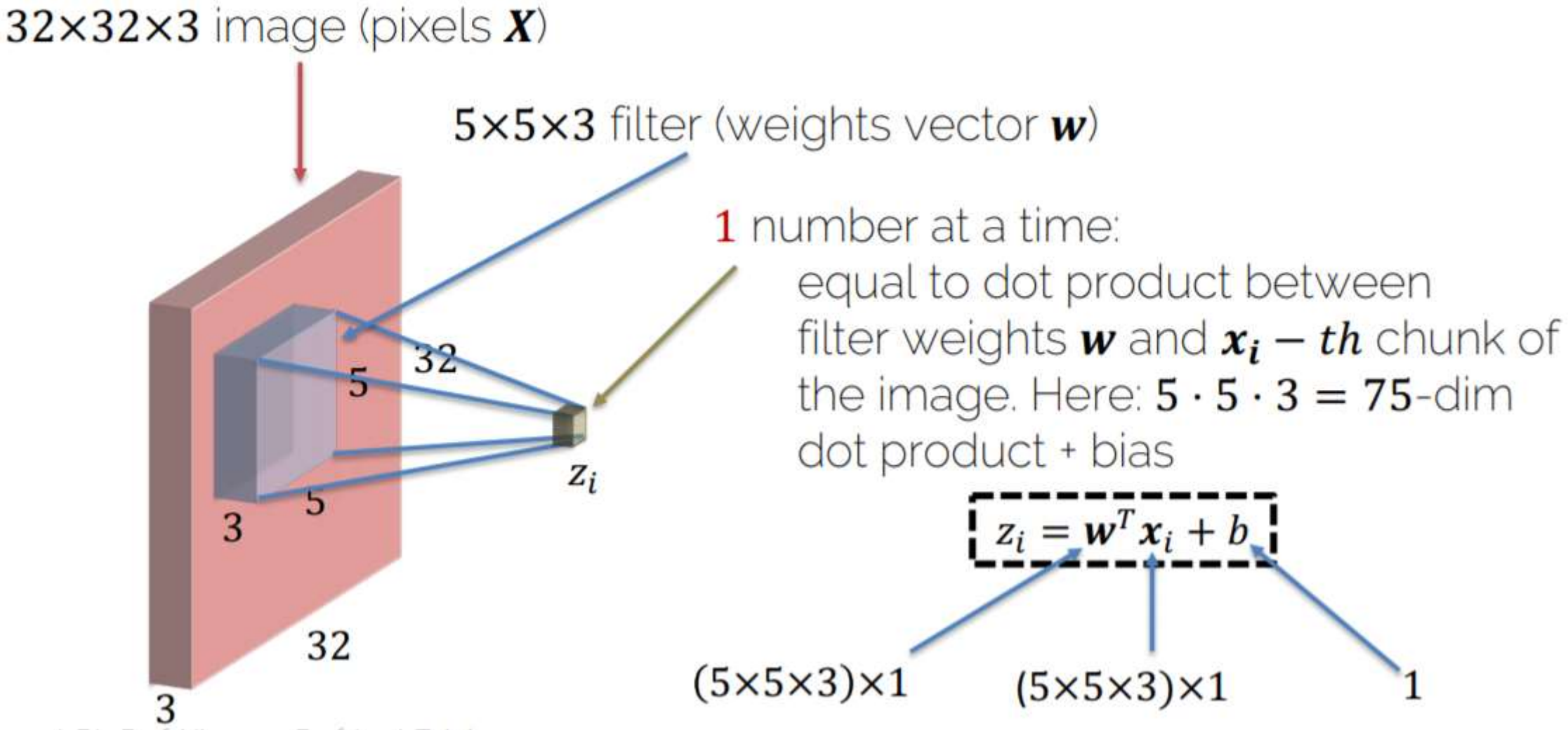
Source: Niessner and Leal-Taixé, Introduction to Deep Learning

Convolutions on RGB Images



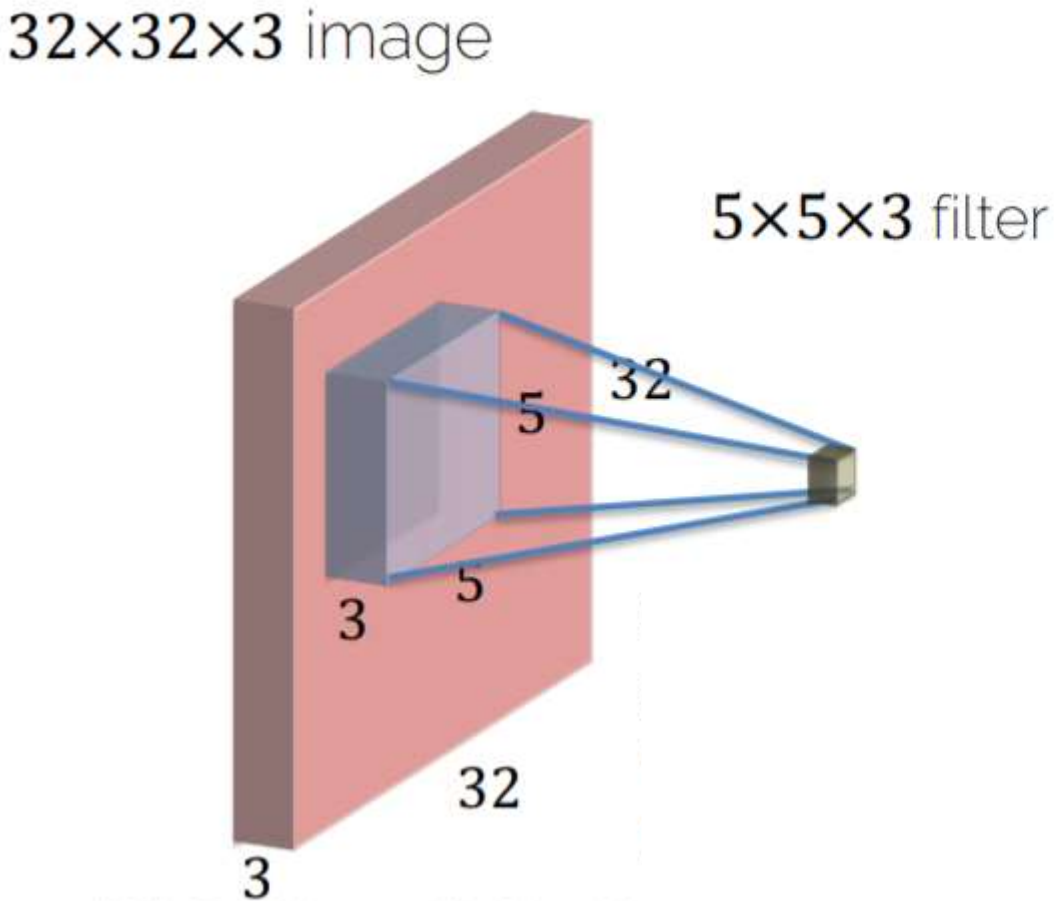
Source: Niessner and Leal-Taixé, Introduction to Deep Learning

Convolutions on RGB Images



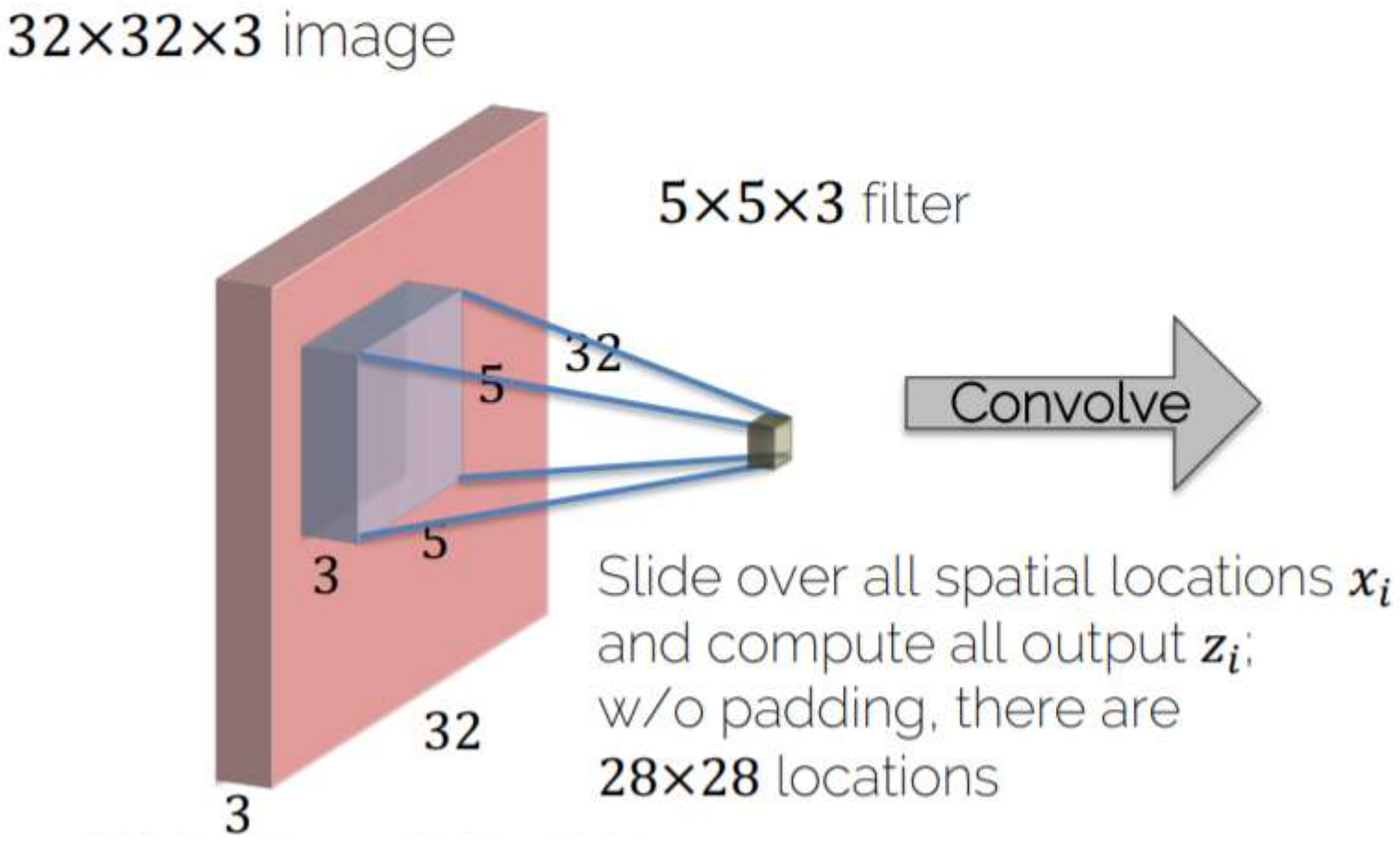
Source: Niessner and Leal-Taixé, Introduction to Deep Learning

Convolutions on RGB Images



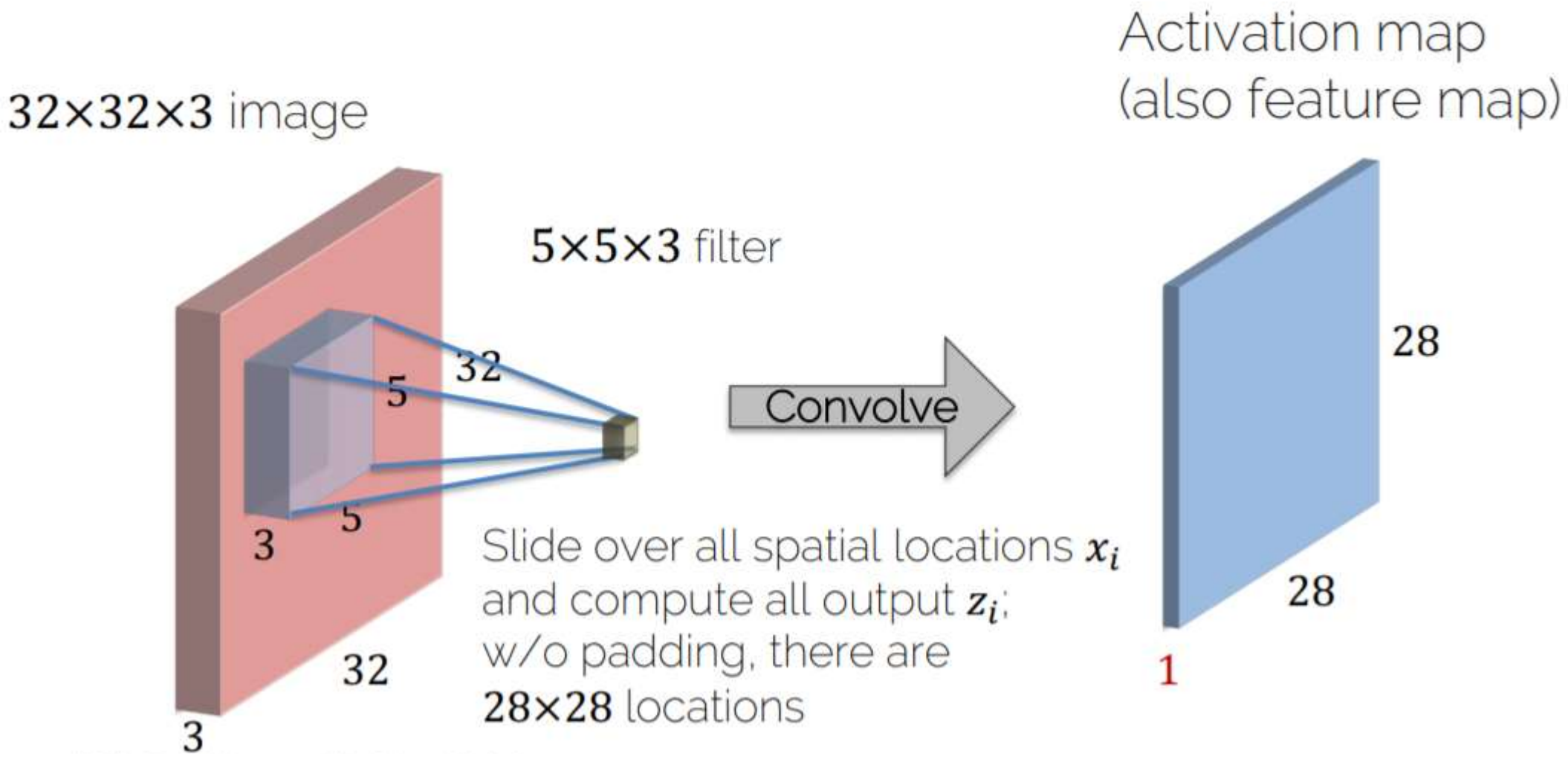
Source: Niessner and Leal-Taixé, Introduction to Deep Learning

Convolutions on RGB Images



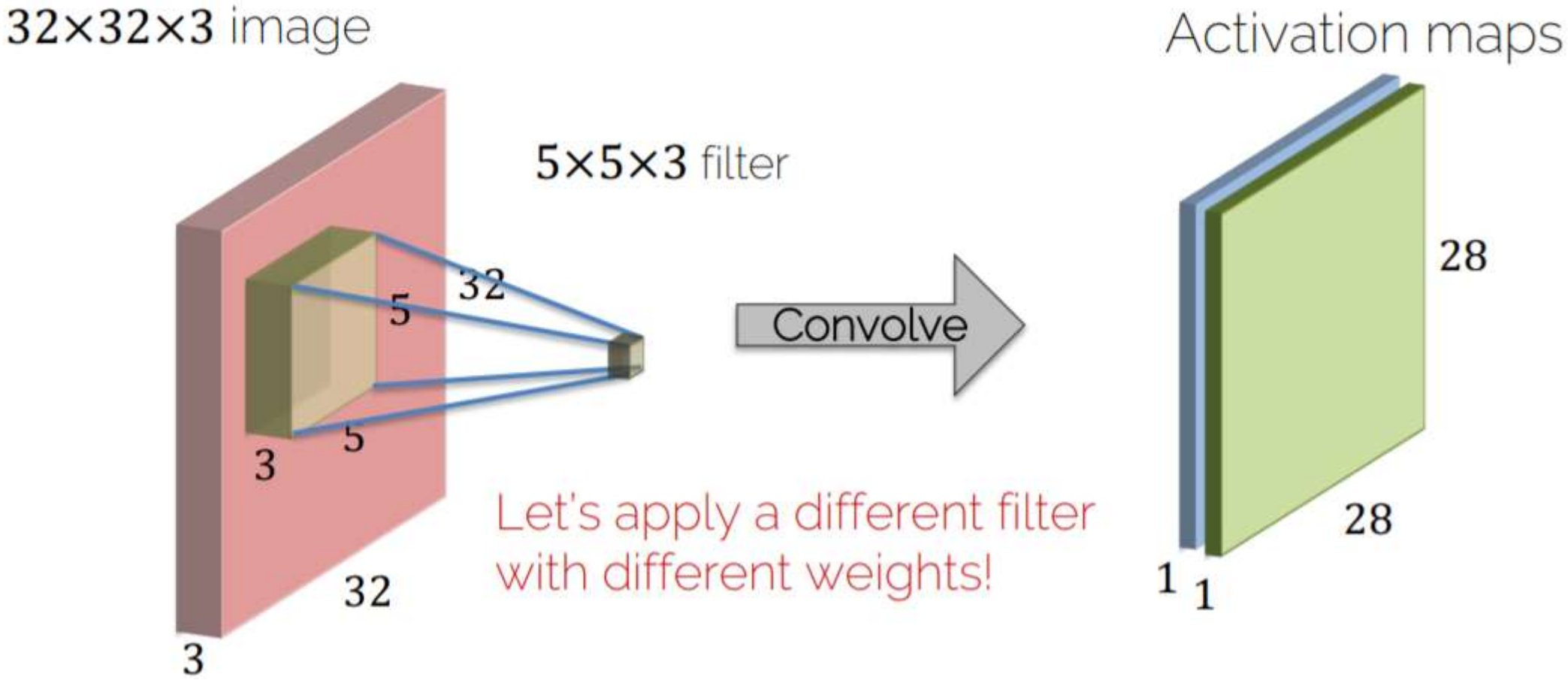
Source: Niessner and Leal-Taixé, Introduction to Deep Learning

Convolutions on RGB Images



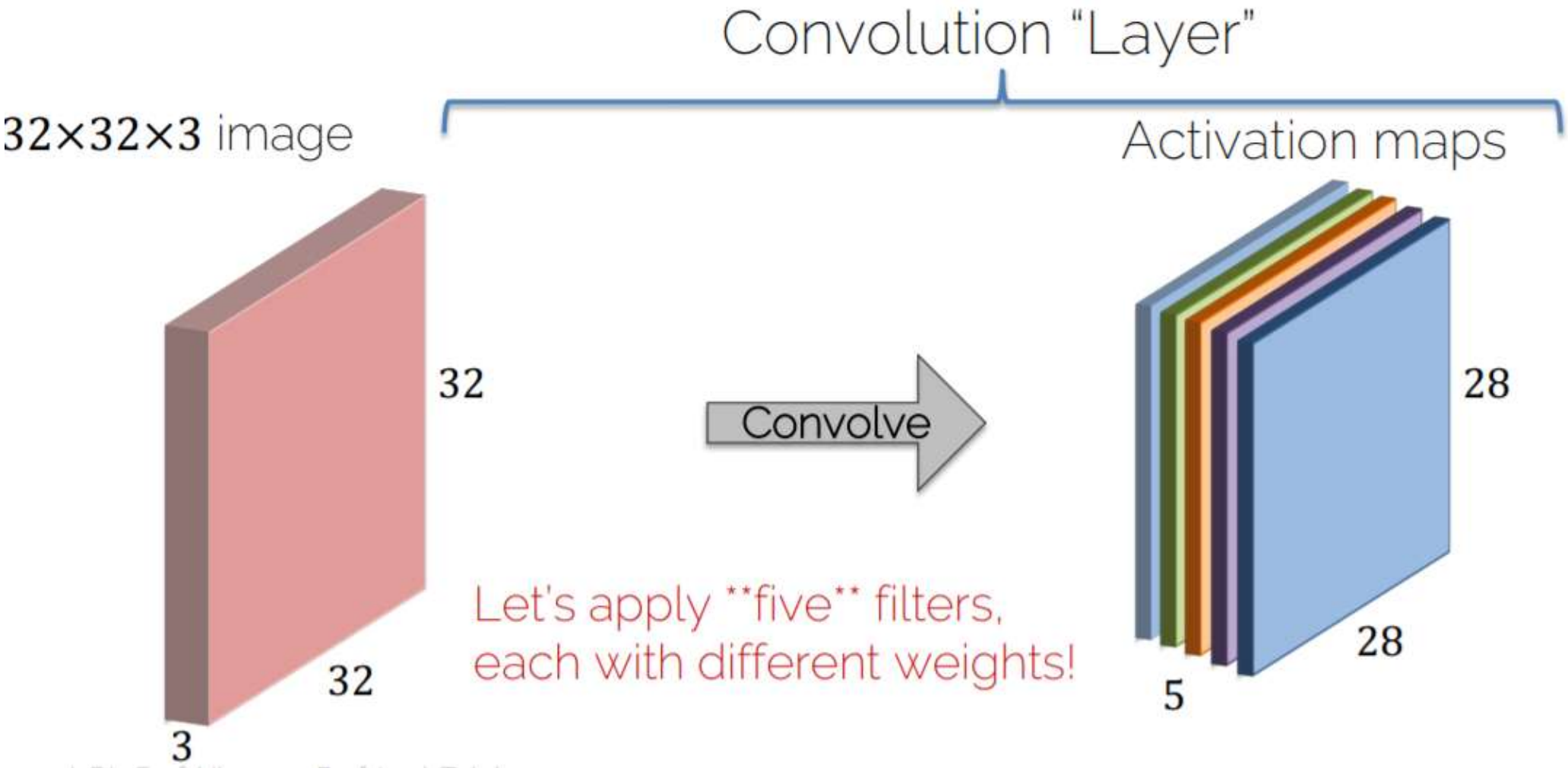
Source: Niessner and Leal-Taixé, Introduction to Deep Learning

Convolution Layer



Source: Niessner and Leal-Taixé, Introduction to Deep Learning

Convolution Layer



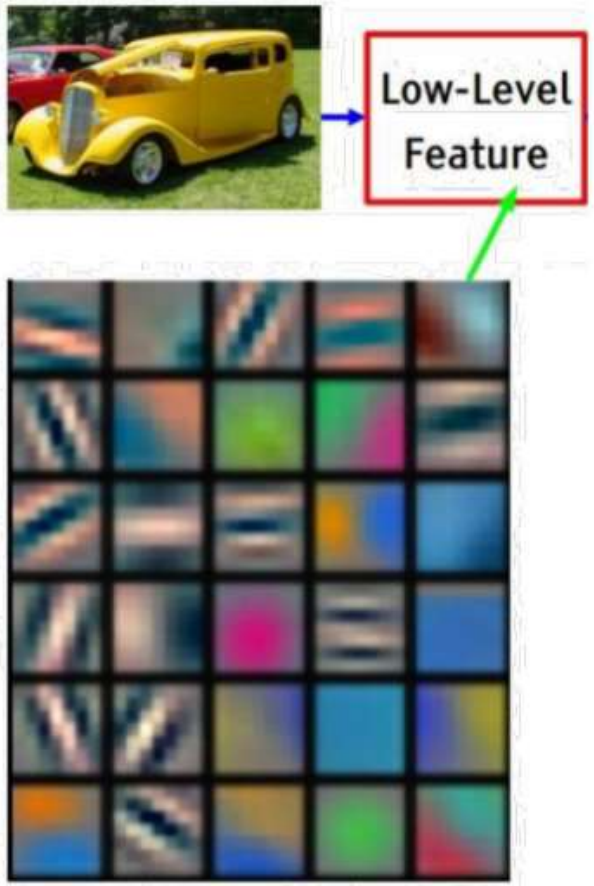
Source: Niessner and Leal-Taixé, Introduction to Deep Learning

Convolution Layer

- A basic layer is defined by
  - Filter width and height (depth is implicitly given)
  - Number of different filter banks (#weight sets)

- Each filter captures a different image characteristic

Different Filters

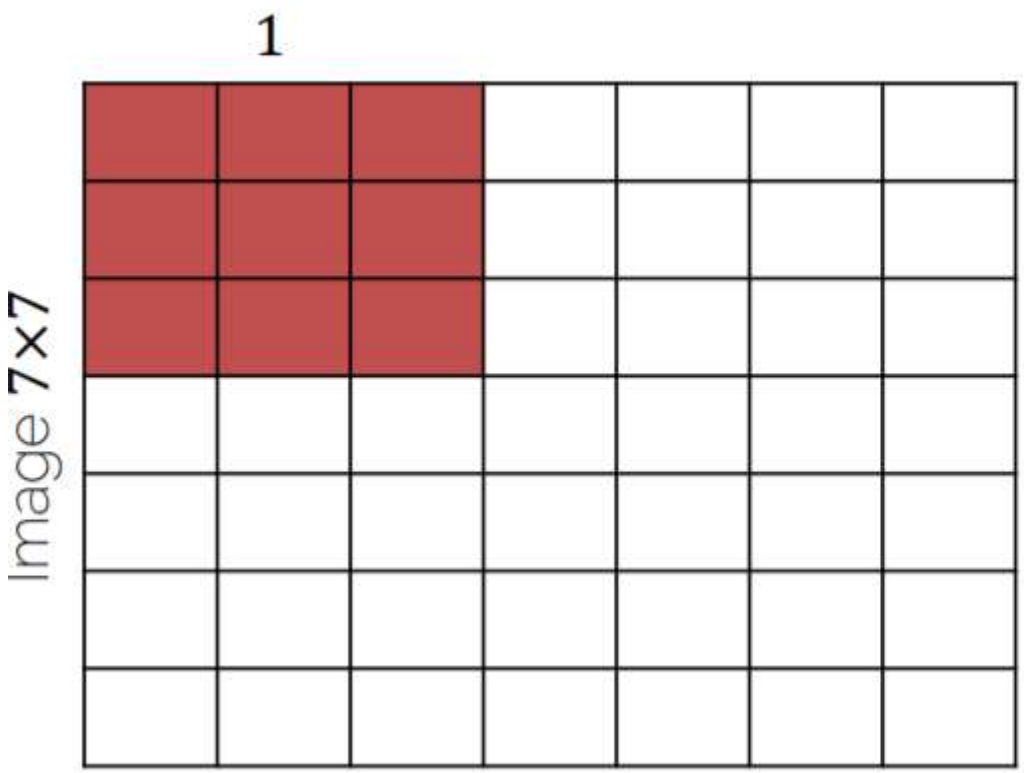


[Zeiler & Fergus, ECCV'14] Visualizing and Understanding Convolutional Networks

- Each filter captures different image characteristics:
  - Horizontal edges
  - Vertical edges
  - Circles
  - Squares
  - …

Source: Niessner and Leal-Taixé, Introduction to Deep Learning

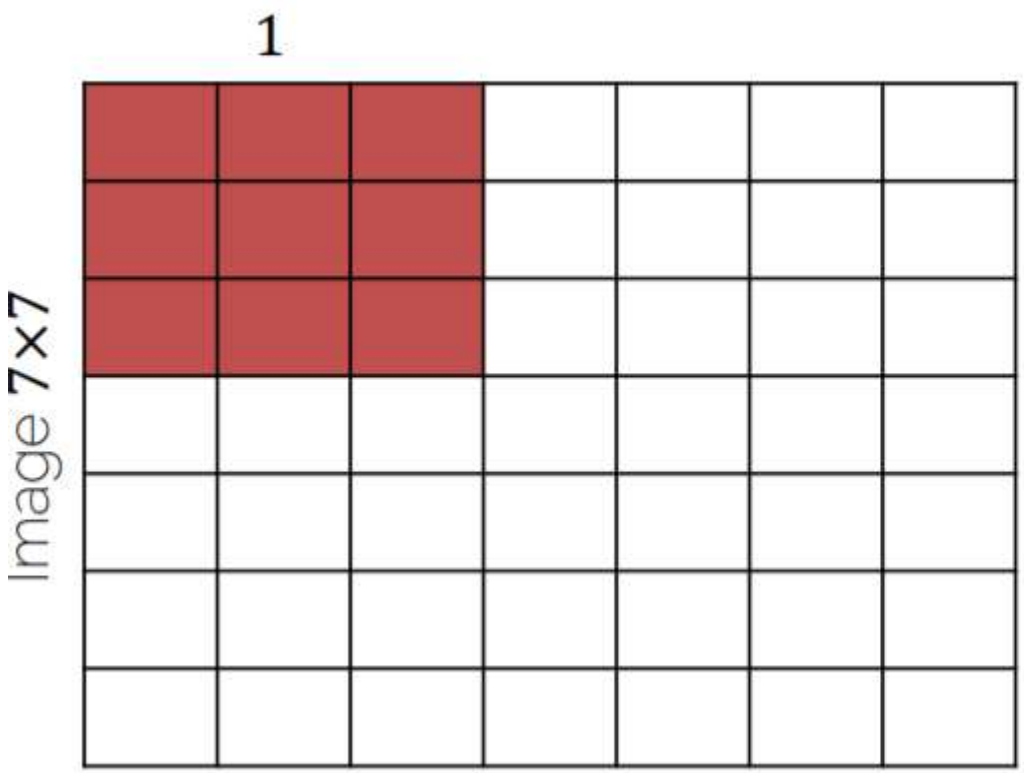
Convolution Layers: Dimensions



Input: 7×7
Filter: 3×3
Output: 5×5

Source: Niessner and Leal-Taixé, Introduction to Deep Learning

Convolution Layers: Dimensions

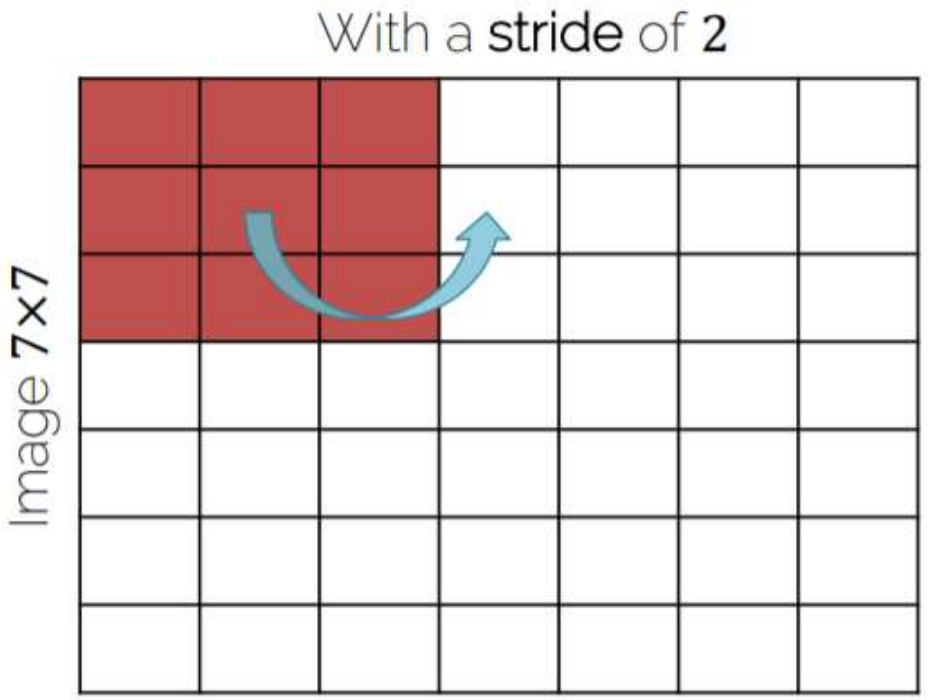


Input:	7×7
Filter:	3×3
Stride:	1
Output:	5×5

Stride of S: apply filter every S-th spatial location; i.e. subsample the image

Source: Niessner and Leal-Taixé, Introduction to Deep Learning

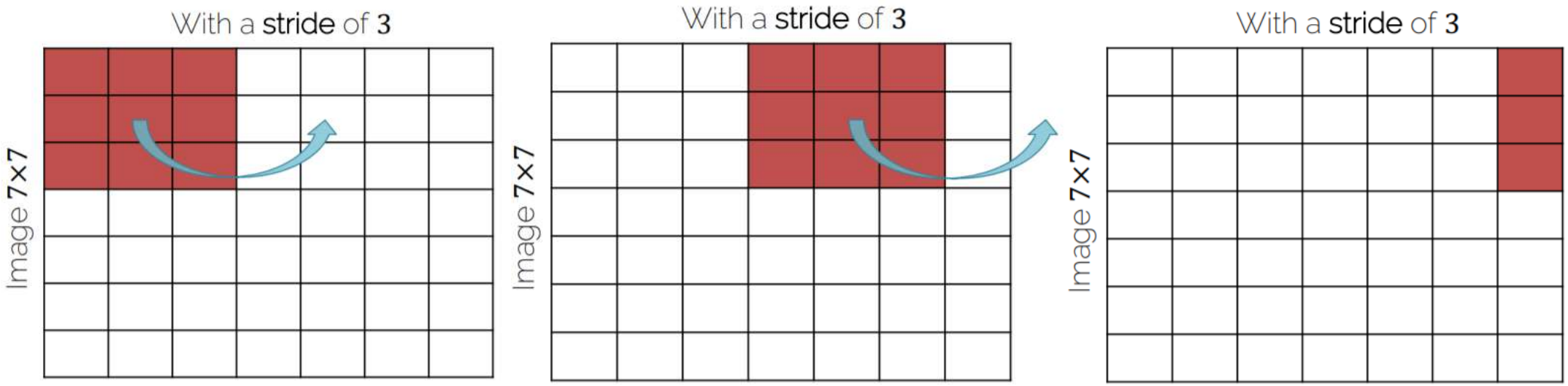
Convolution Layers: Stride



Input:	7×7
Filter:	3×3
Stride:	2
Output:	3×3

Source: Niessner and Leal-Taixé, Introduction to Deep Learning

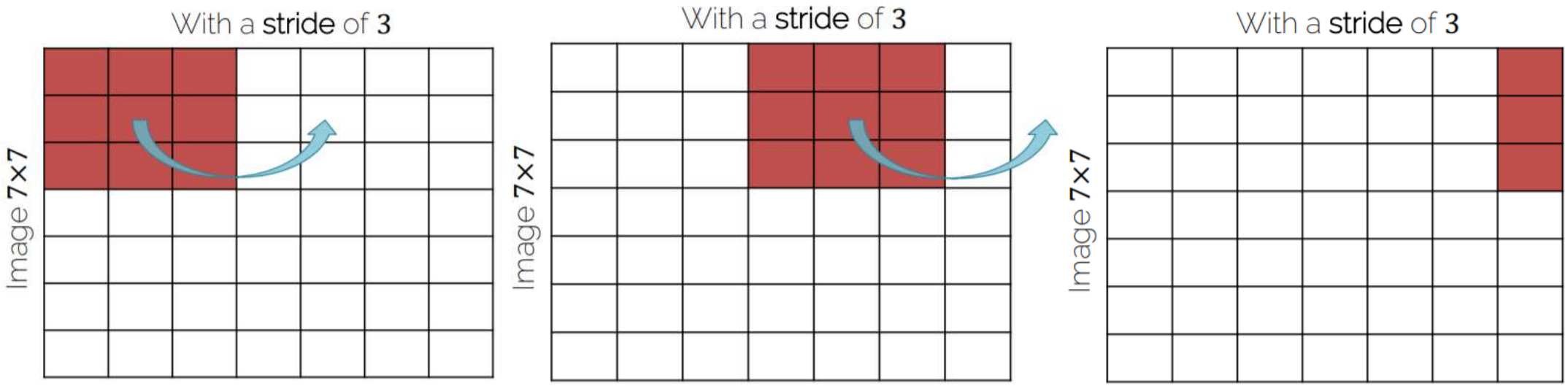
Convolution Layers: Stride



Input:	7×7
Filter:	3×3
Stride:	3
Output:	? × ?

Source: Niessner and Leal-Taixé, Introduction to Deep Learning

Convolution Layers: Stride



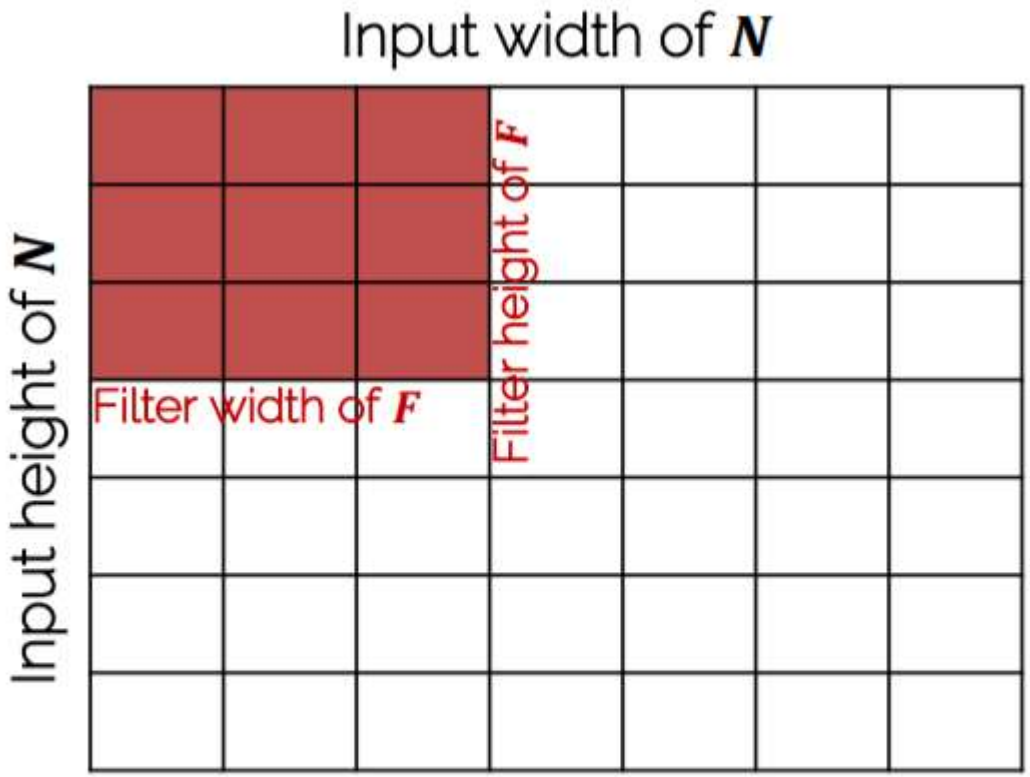
Input:	7×7
Filter:	3×3
Stride:	3
Output:	? × ?

Does not really fit (remainder left)

→ Illegal stride for input & filter size!

Source: Niessner and Leal-Taixé, Introduction to Deep Learning

Convolution Layers: Dimensions



Input: $N \times N$

Filter: $F \times F$

Stride: S

Output: $\left(\frac{N-F}{S}+1\right) \times \left(\frac{N-F}{S}+1\right)$

$N = 7, F = 3, S = 1$: $\frac{7-3}{1}+1 = 5$

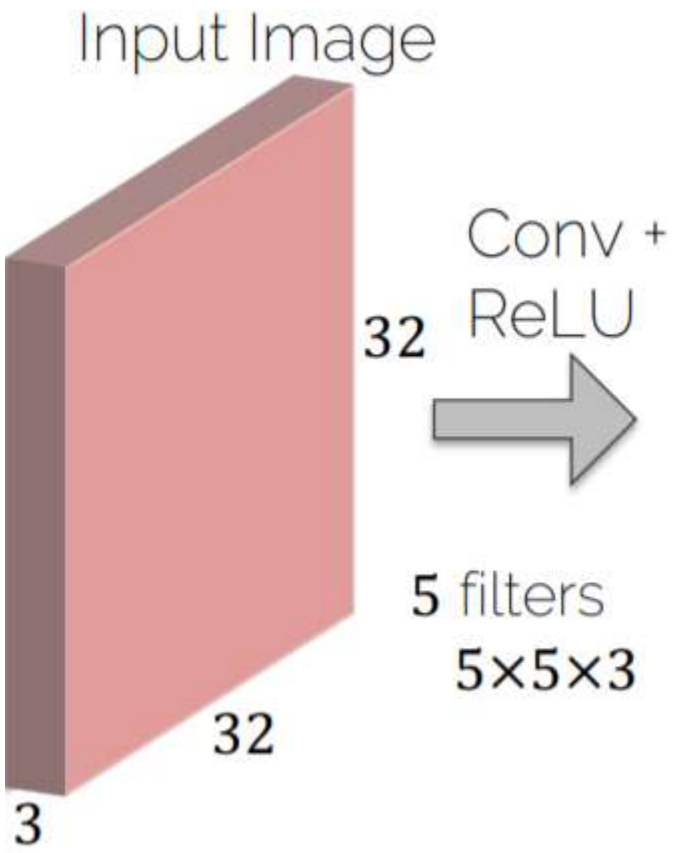
$N = 7, F = 3, S = 2$: $\frac{7-3}{2}+1 = 3$

$N = 7, F = 3, S = 3$: $\frac{7-3}{3}+1 = 2.\bar{3}$

Fractions are illegal

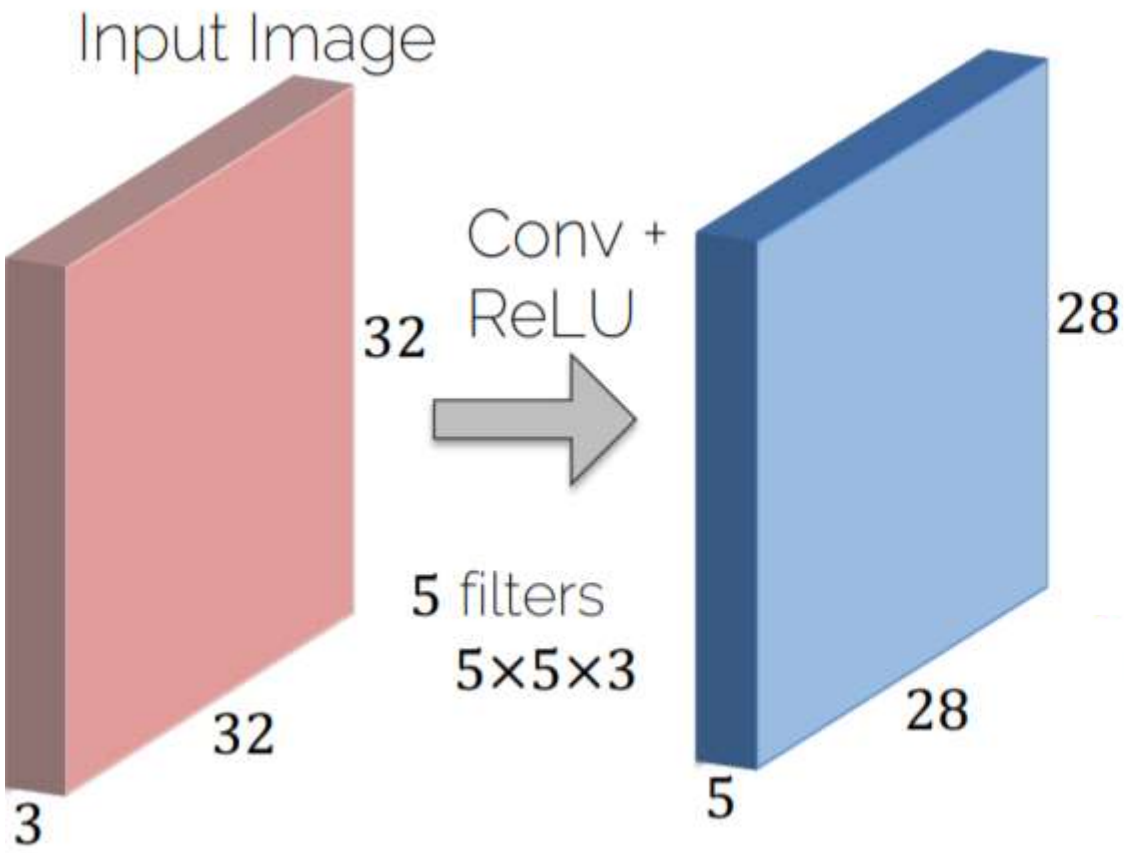
Source: Niessner and Leal-Taixé, Introduction to Deep Learning

Convolution Layers: Dimensions



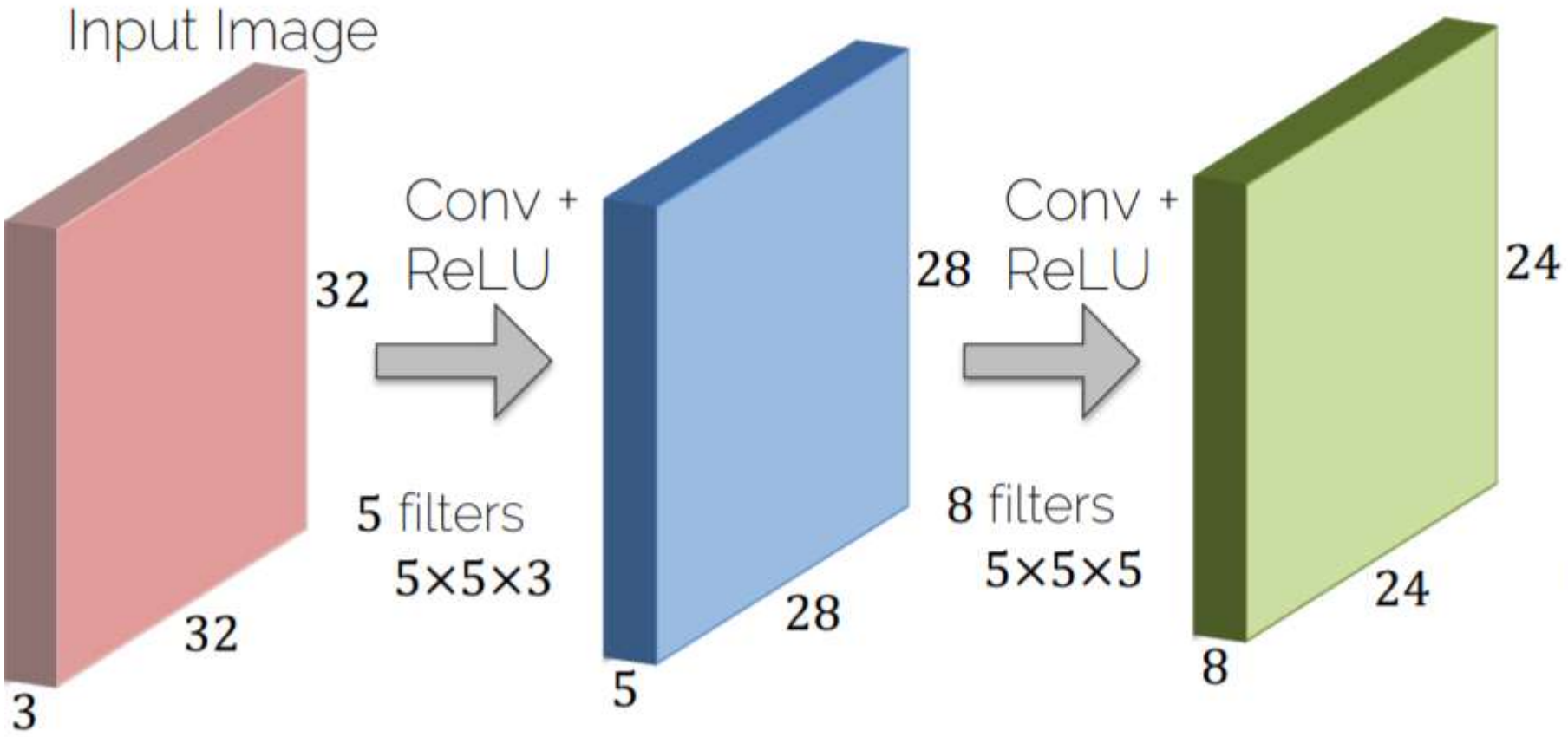
Source: Niessner and Leal-Taixé, Introduction to Deep Learning

Convolution Layers: Dimensions



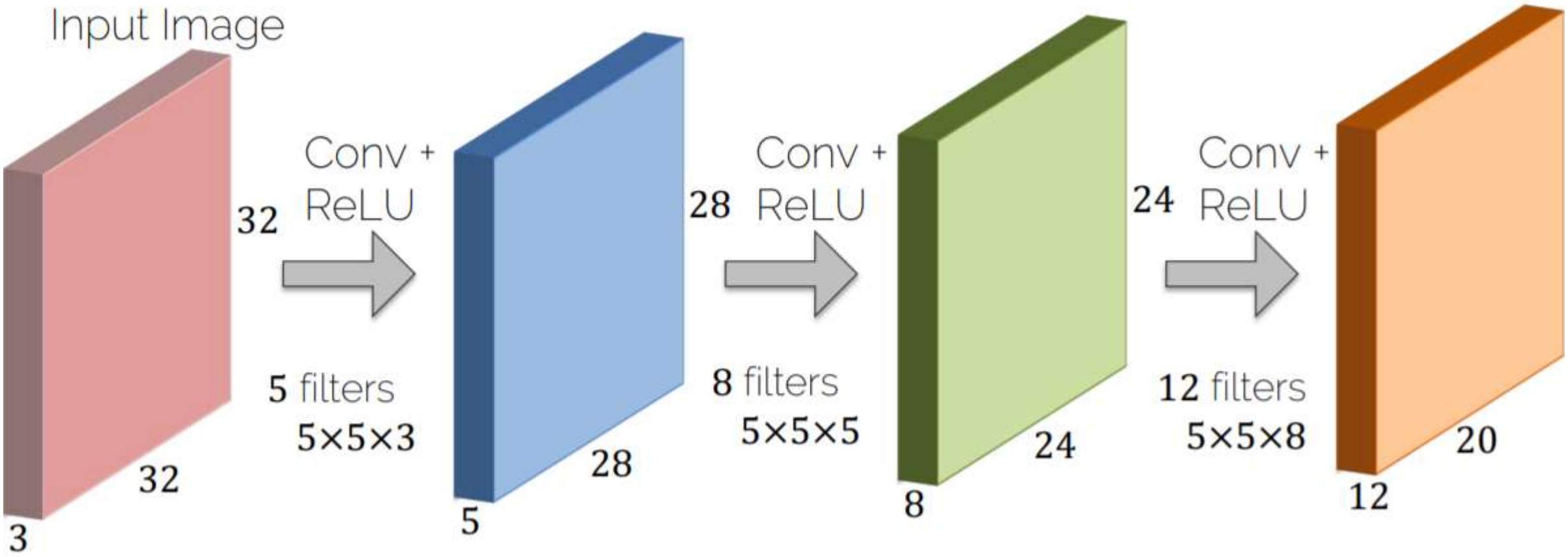
Source: Niessner and Leal-Taixé, Introduction to Deep Learning

Convolution Layers: Dimensions



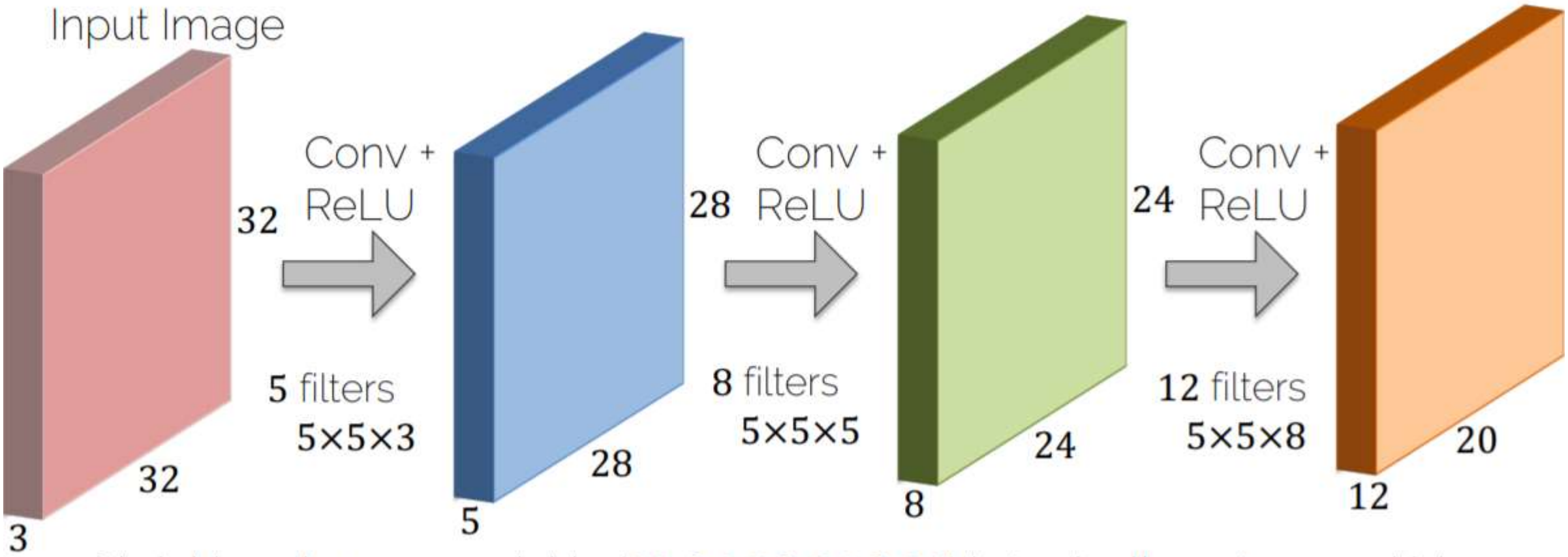
Source: Niessner and Leal-Taixé, Introduction to Deep Learning

Convolution Layers: Dimensions



Source: Niessner and Leal-Taixé, Introduction to Deep Learning

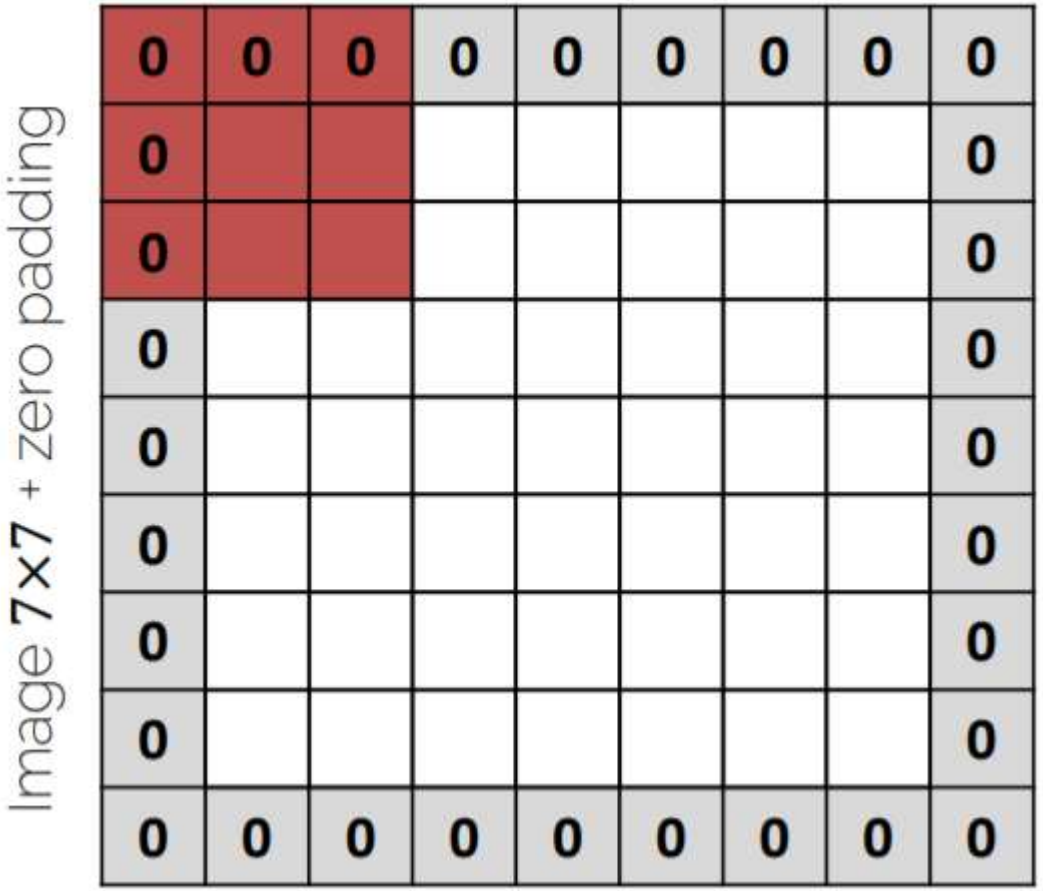
Convolution Layers: Dimensions



Shrinking down so quickly (32→28→24→20) is typically not a good idea…

Source: Niessner and Leal-Taixé, Introduction to Deep Learning

Convolution Layers: Padding

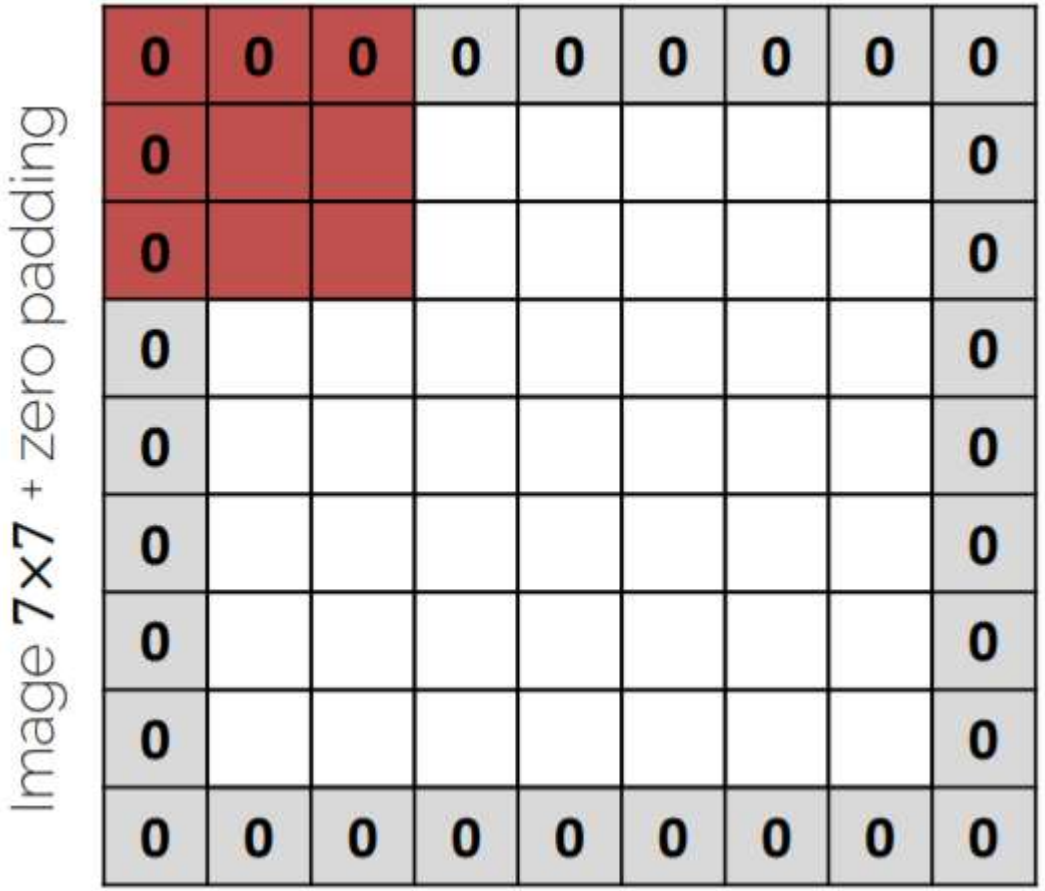


Why padding?

- Sizes get small too quickly
- Corner pixel is only used once

Source: Niessner and Leal-Taixé, Introduction to Deep Learning

Convolution Layers: Padding



Input ($N \times N$): 7×7
Filter ($F \times F$): 3×3
Padding (P): 1
Stride (S): 1
Output 7×7

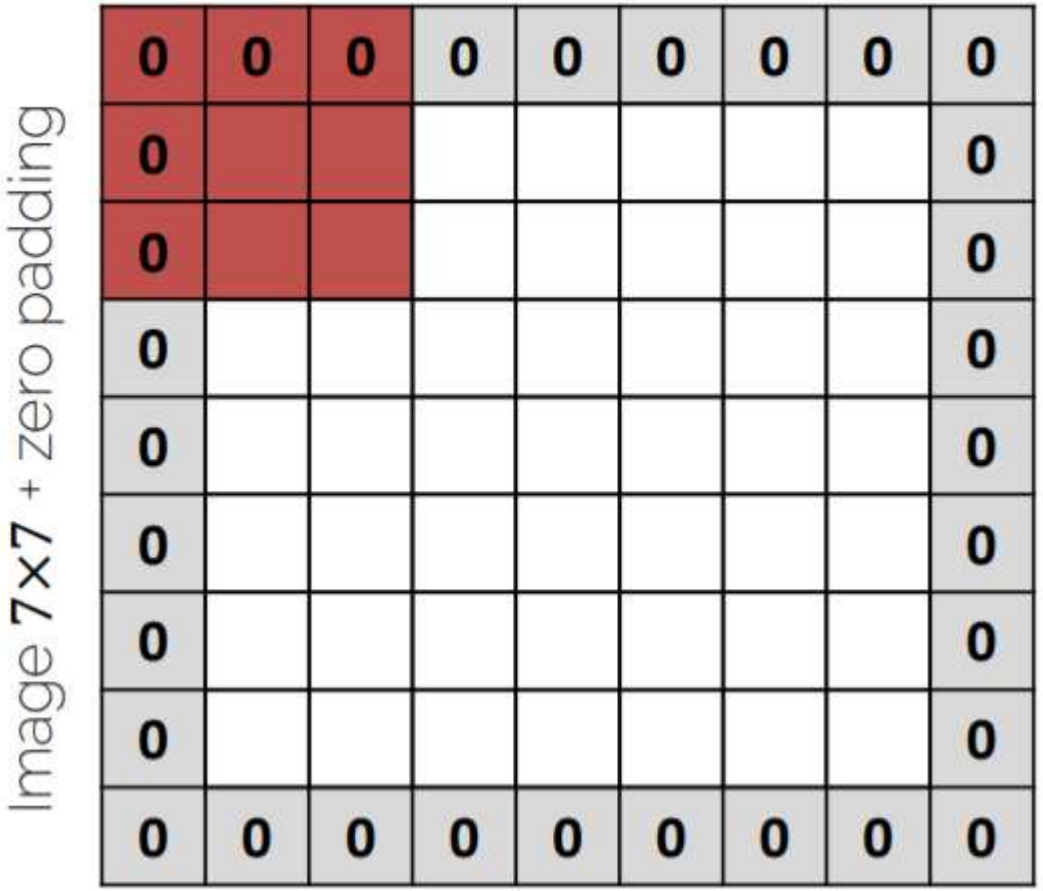
Most common is '*zero*' padding

Output Size:

$$\left(\frac{N+2 \cdot P-F}{S}+1\right) \times \left(\frac{N+2 \cdot P-F}{S}+1\right)$$

Source: Niessner and Leal-Taixé, Introduction to Deep Learning

Convolution Layers: Padding



Types of convolutions:

- **Valid convolution**: using no padding
- **Same convolution**: output=input size

Set padding to $P = \frac{F-1}{2}$

Source: Niessner and Leal-Taixé, Introduction to Deep Learning

Convolution Layers: Dimensions

Example

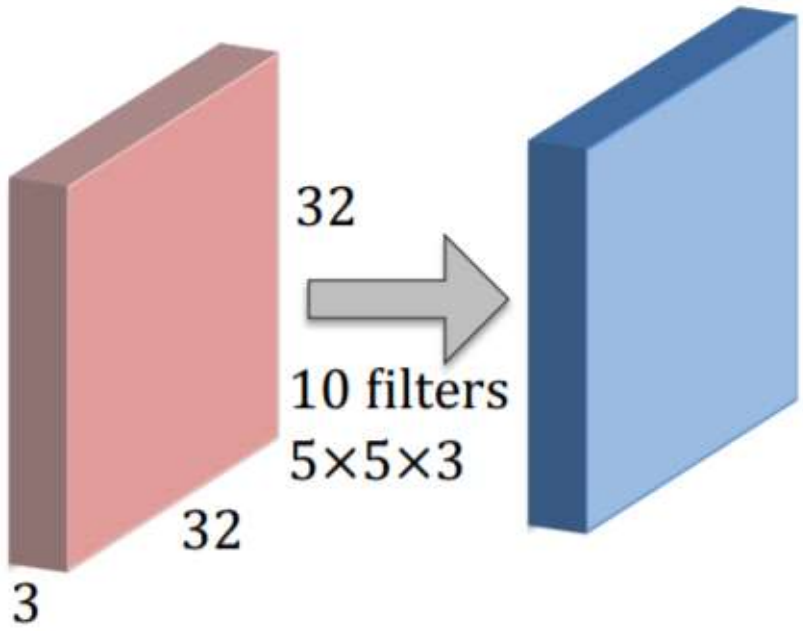
Input image: 32×32×3

10 filters 5×5

Stride 1

Pad 2

Depth of 3 is implicitly given



Source: Niessner and Leal-Taixé, Introduction to Deep Learning

Convolution Layers: Dimensions

Example

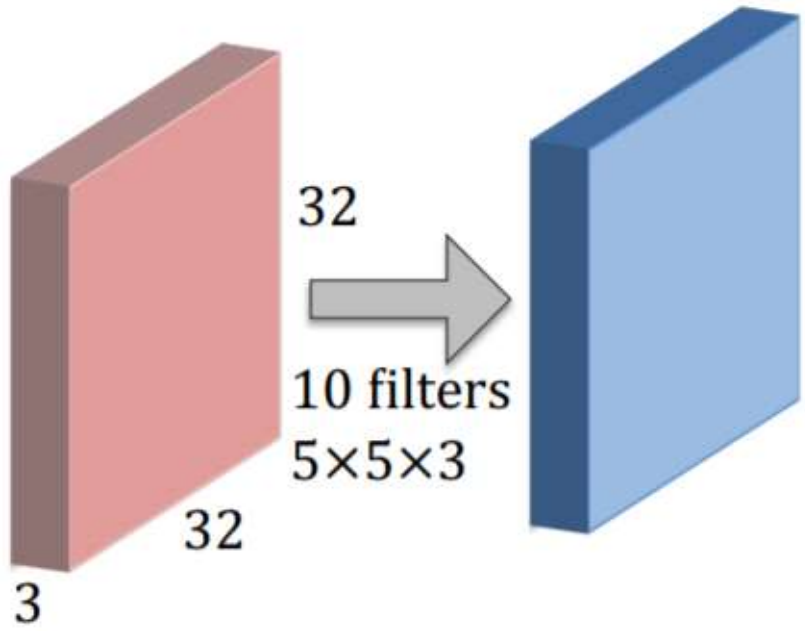
Input image: 32×32×3

10 filters 5×5

Stride 1

Pad 2

Depth of 3 is implicitly given



Remember

Output: $\left(\left\lfloor\frac{N+2\cdot P-F}{S}\right\rfloor+1\right)\times\left(\left\lfloor\frac{N+2\cdot P-F}{S}\right\rfloor+1\right)$

Source: Niessner and Leal-Taixé, Introduction to Deep Learning

Convolution Layers: Dimensions

Example

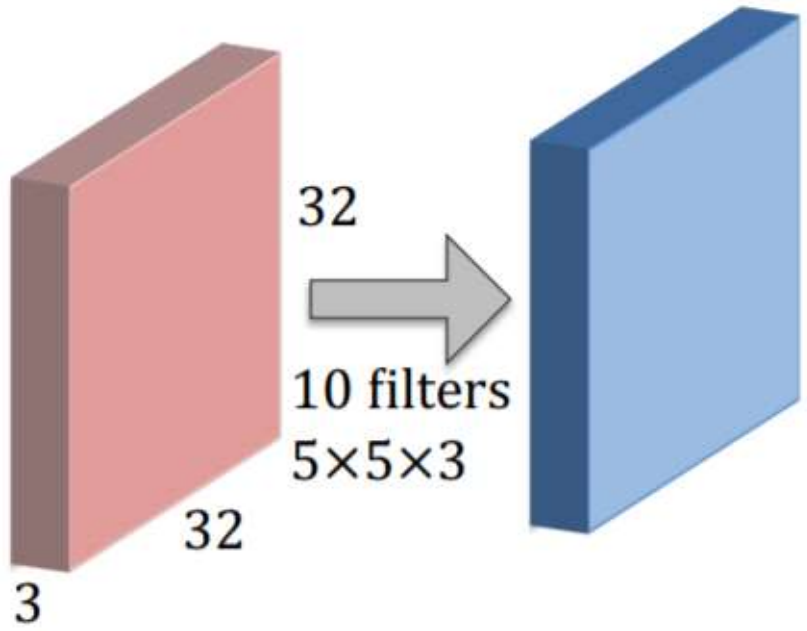
Input image: 32×32×3

10 filters 5×5

Stride 1

Pad 2

Depth of 3 is implicitly given



Output size is:

$$\frac{32 + 2 \cdot 2 - 5}{1} + 1 = 32$$

i.e. 32×32×10

Remember

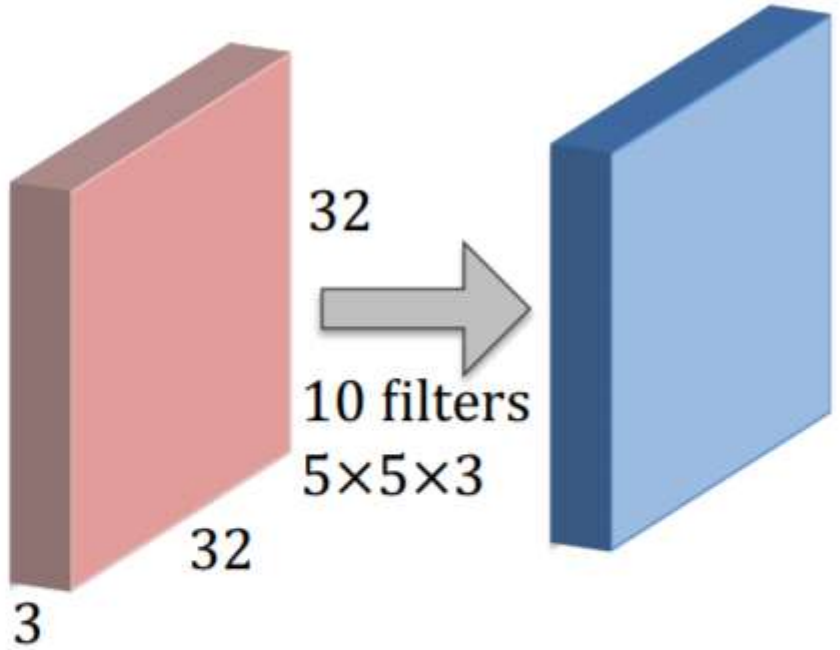
Output: $\left(\left\lfloor\frac{N+2\cdot P-F}{S}\right\rfloor + 1\right) \times \left(\left\lfloor\frac{N+2\cdot P-F}{S}\right\rfloor + 1\right)$

Source: Niessner and Leal-Taixé, Introduction to Deep Learning

Convolution Layers: Dimensions

Example

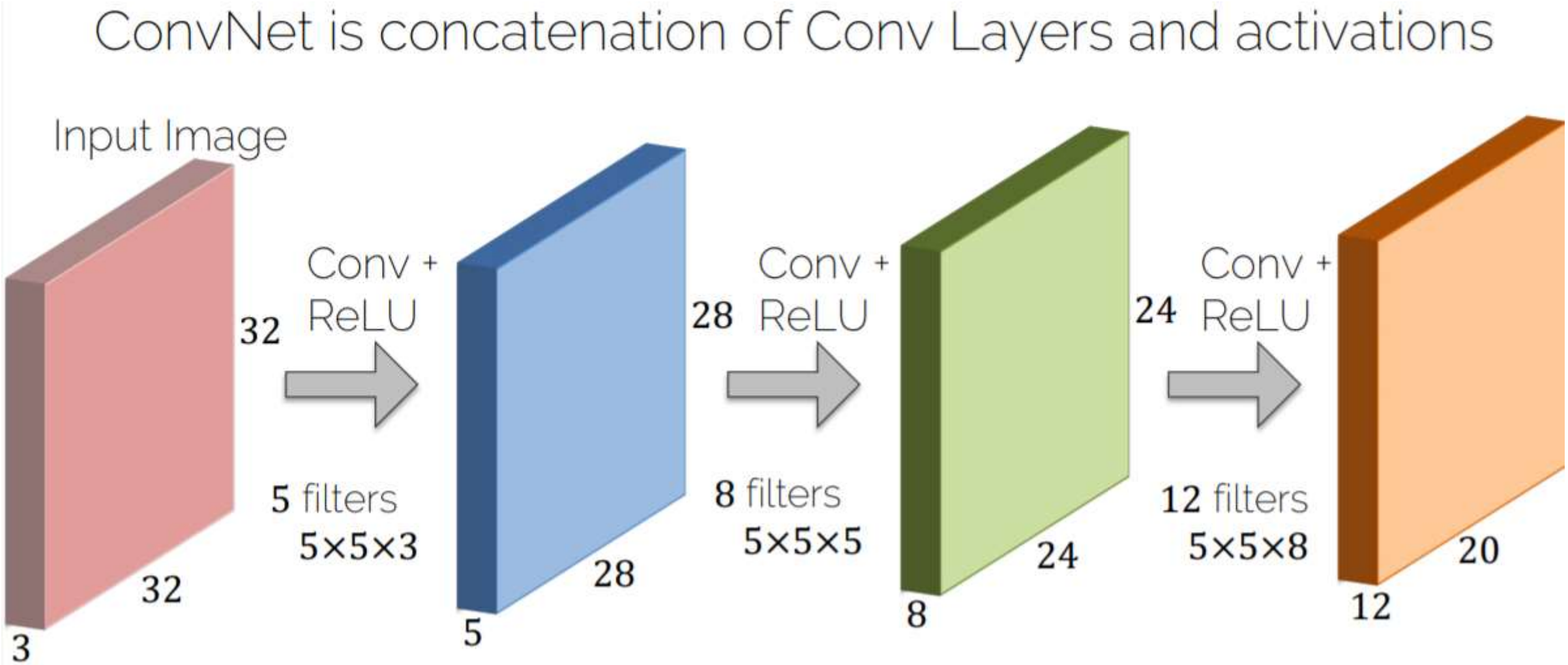
Input image: $32 \times 32 \times 3$
10 filters 5×5
Stride 1
Pad 2



Number of parameters (weights):
Each filter has $5 \times 5 \times 3 + 1 = 76$ params (+1 for bias)
-> $76 \cdot 10 = 760$ parameters in layer

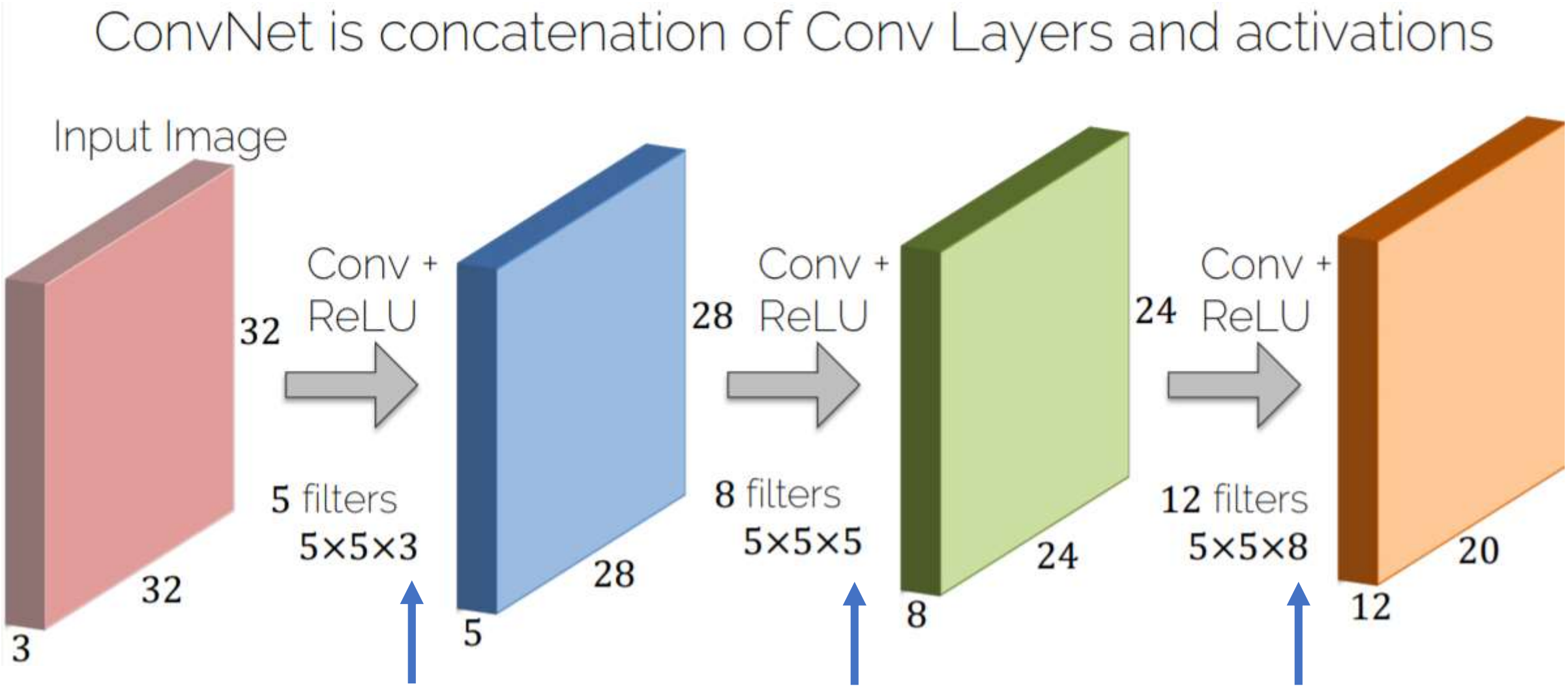
Source: Niessner and Leal-Taixé, Introduction to Deep Learning

CNN Prototype



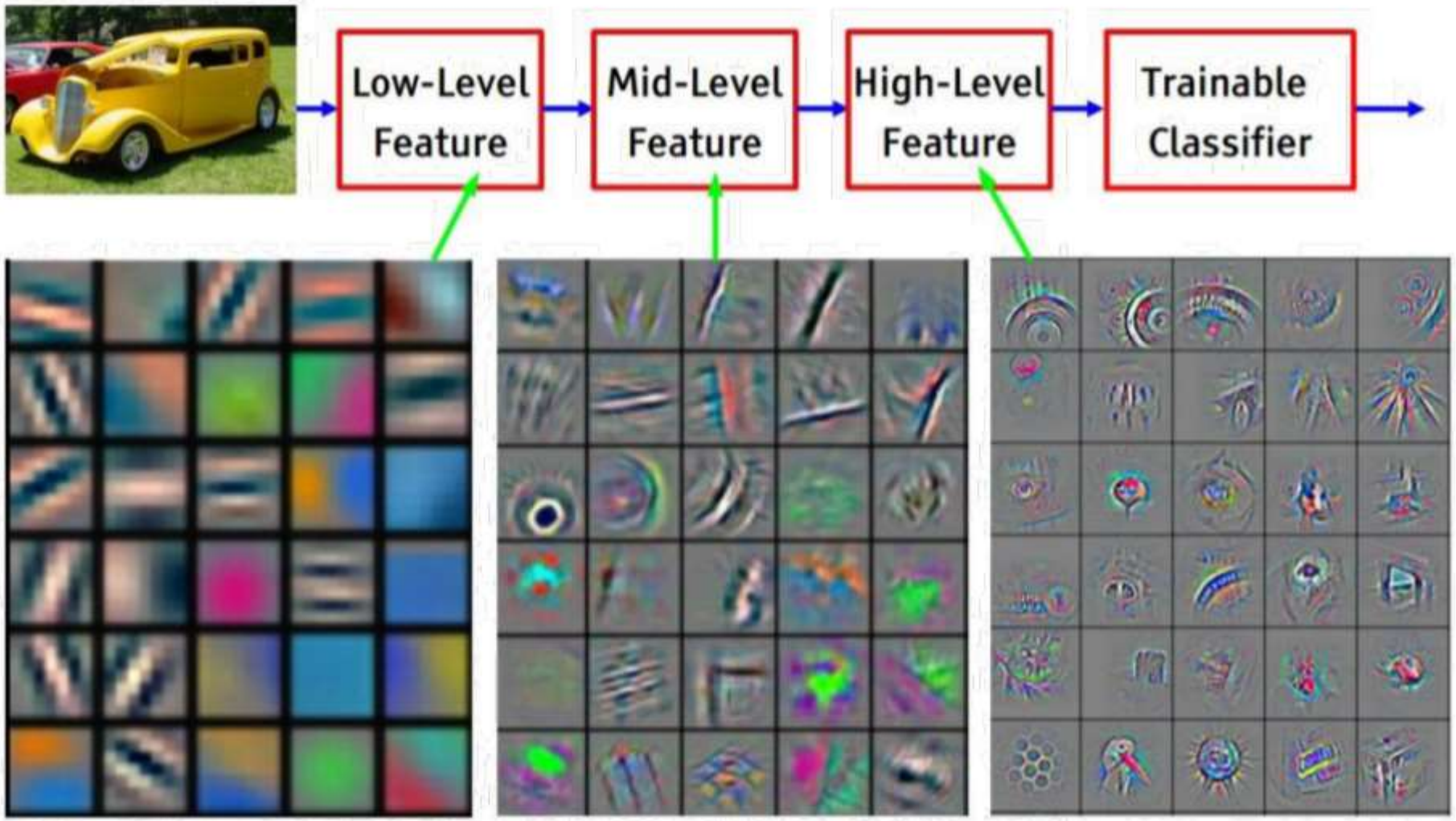
Source: Niessner and Leal-Taixé, Introduction to Deep Learning

CNN Prototype



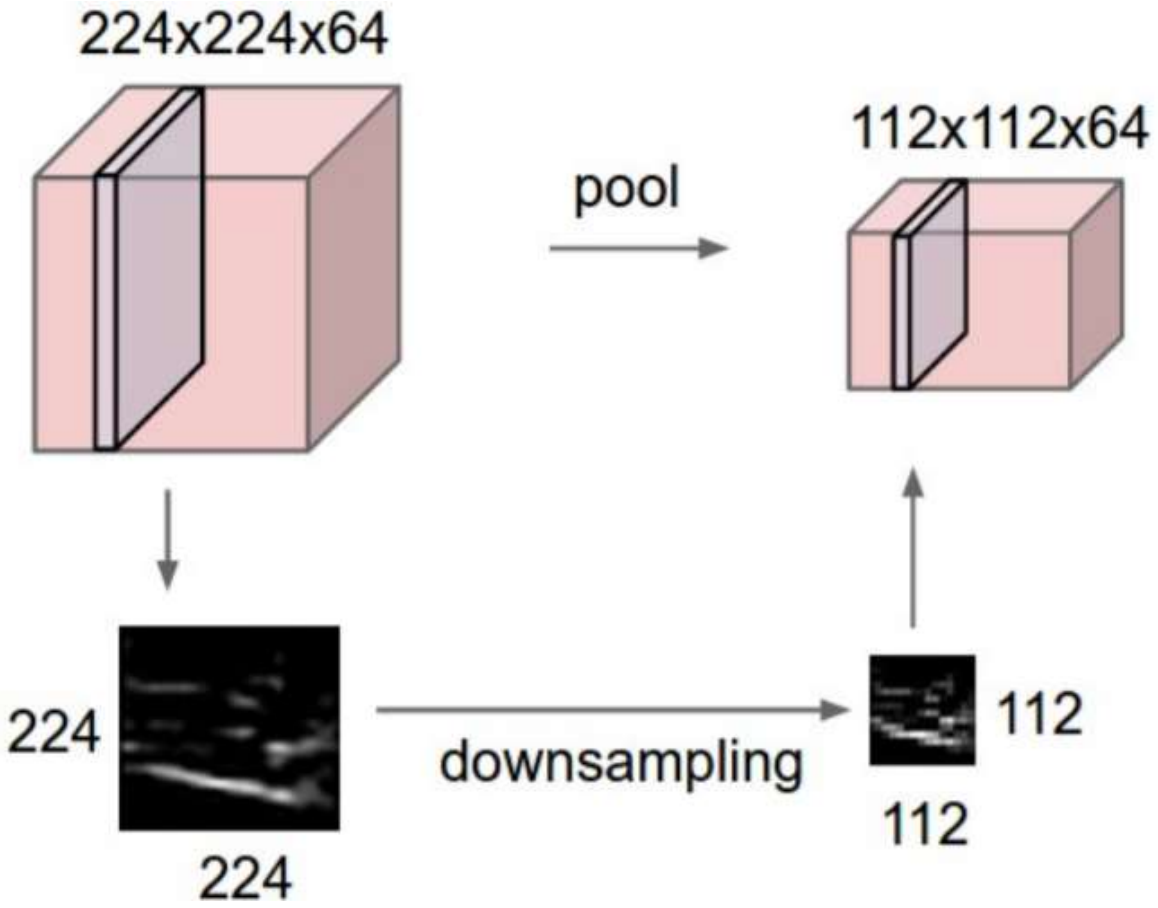
Source: Niessner and Leal-Taixé, Introduction to Deep Learning

CNN Learned Filters



[Zeiler & Fergus, ECCV'14] Visualizing and Understanding Convolutional Networks

Pooling Layer



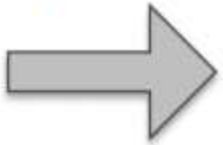
- makes the representations smaller and more manageable
- operates over each activation map independently:

Pooling Layer: Max Pooling

Single depth slice of input

3	1	3	5
6	0	7	9
3	2	1	4
0	2	4	3

Max pool with
2×2 filters and stride 2



'Pooled' output

6	9
3	4

Source: Niessner and Leal-Taixé, Introduction to Deep Learning

Pooling Layer

- Input is a volume of size $W_{in} \times H_{in} \times D_{in}$
- Two hyperparameters
  - Spatial filter extent F
  - Stride S
- Output volume is of size $W_{out} \times H_{out} \times D_{out}$
  - $W_{out} = \frac{W_{in} - F}{S} + 1$
  - $H_{out} = \frac{H_{in} - F}{S} + 1$
  - $D_{out} = D_{in}$
- Does not contain parameters; e.g. it's fixed function

Source: Niessner and Leal-Taixé, Introduction to Deep Learning

Pooling Layer

- Input is a volume of size $W_{in} \times H_{in} \times D_{in}$
- Two hyperparameters
  - Spatial filter extent F
  - Stride S
- Output volume is of size $W_{out} \times H_{out} \times D_{out}$
  - $W_{out} = \frac{W_{in} - F}{S} + 1$
  - $H_{out} = \frac{H_{in} - F}{S} + 1$
  - $D_{out} = D_{in}$
- Does not contain parameters; e.g. it's fixed function

Common settings:
$F = 2, S = 2$
$F = 3, S = 2$

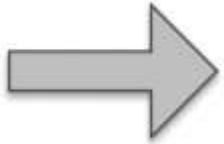
Source: Niessner and Leal-Taixé, Introduction to Deep Learning

Pooling Layer: Average Pooling

Single depth slice of input

3	1	3	5
6	0	7	9
3	2	1	4
0	2	4	3

Average pool with 2×2 filters and stride 2



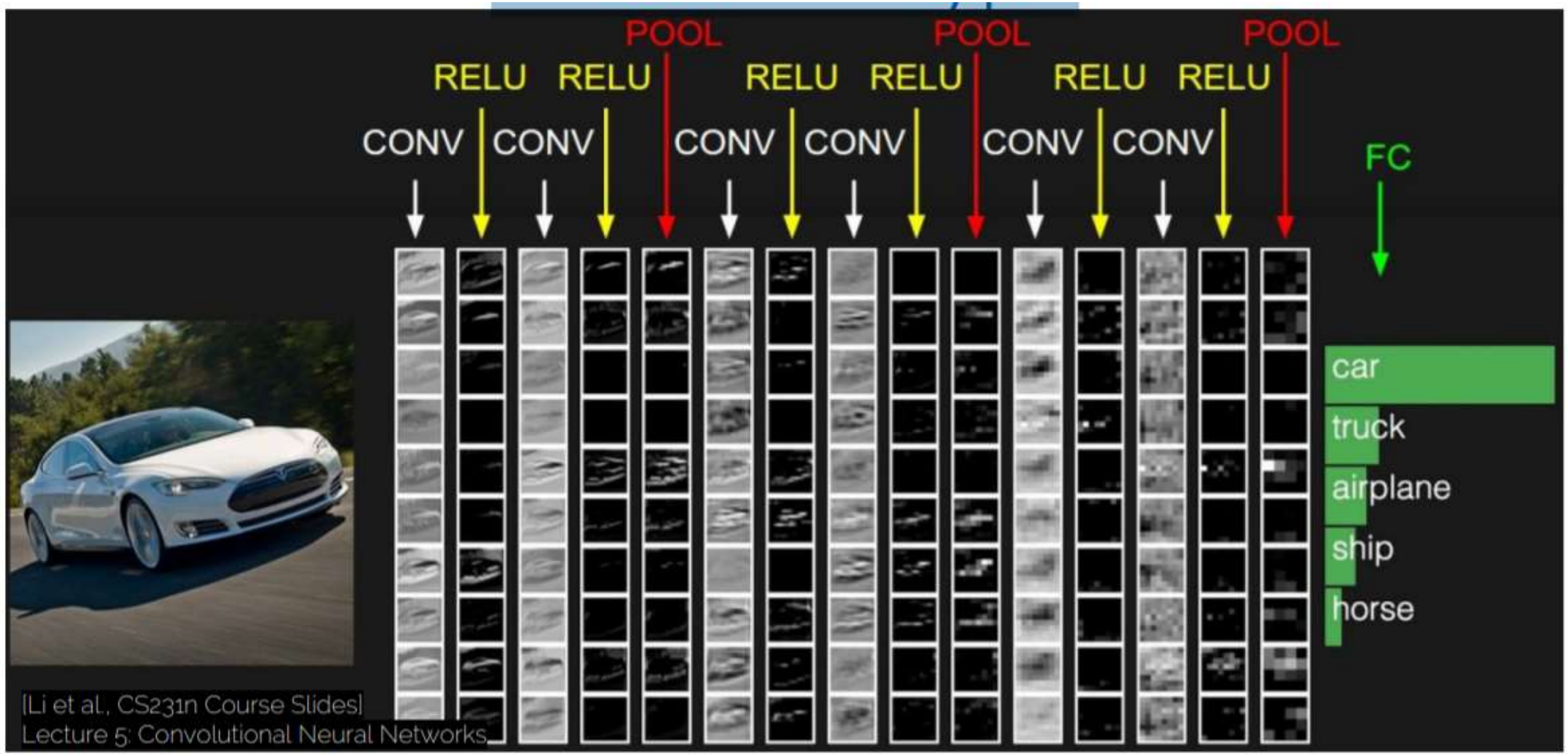
'Pooled' output

2.5	6
1.75	3

Typically used deeper in the network

Source: Niessner and Leal-Taixé, Introduction to Deep Learning

CNN Prototype

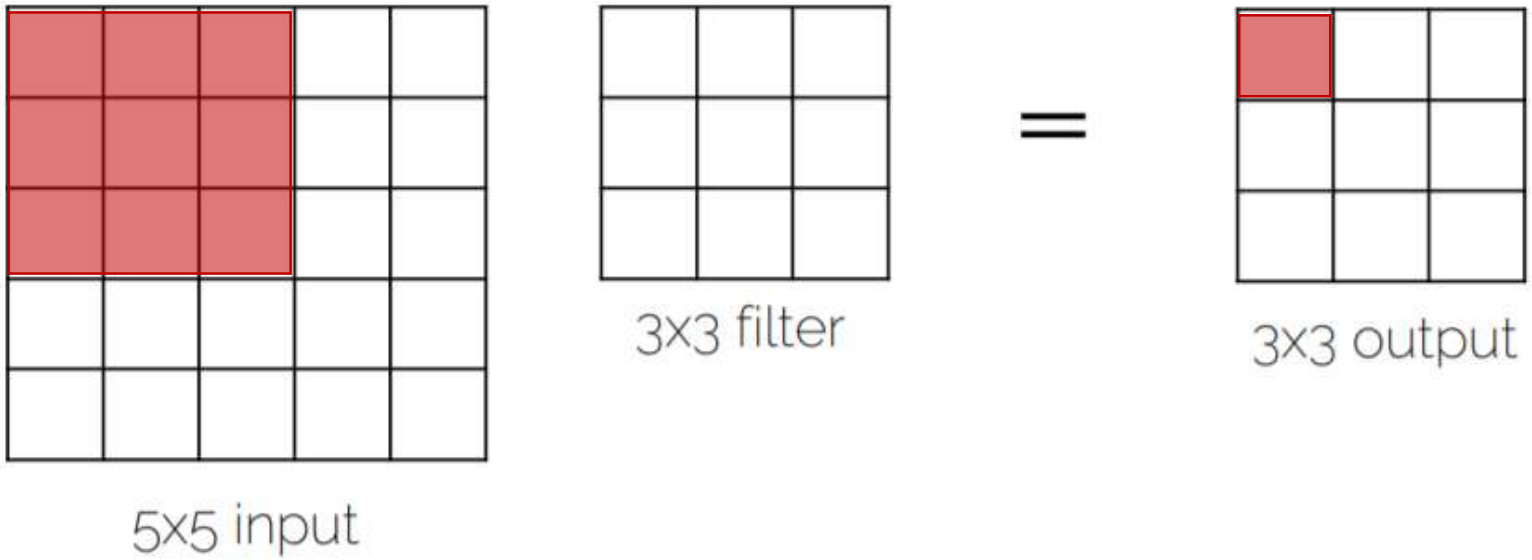


Final Fully-Connected Layer

- Same as what we had in ‘ordinary’ neural networks
  - Make the final decision with the extracted features from the convolutions
  - One or two FC layers typically

Receptive Field

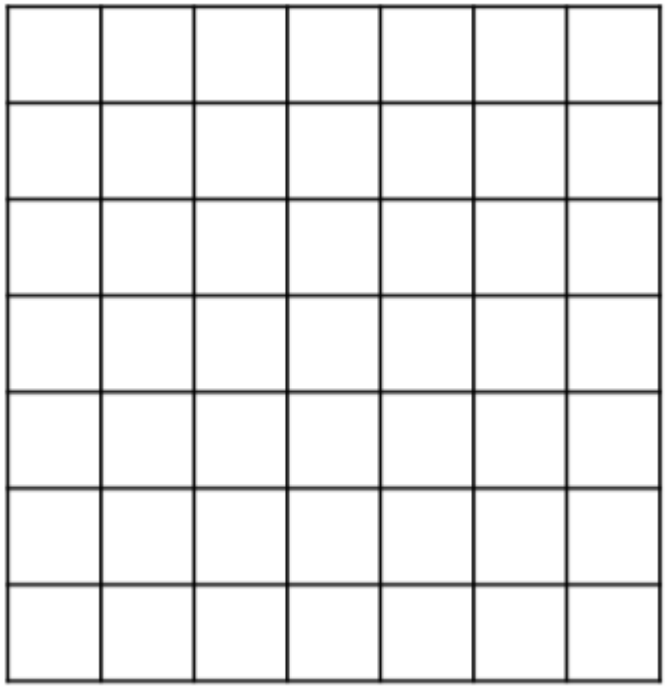
Spatial extent of the connectivity of a convolutional filter

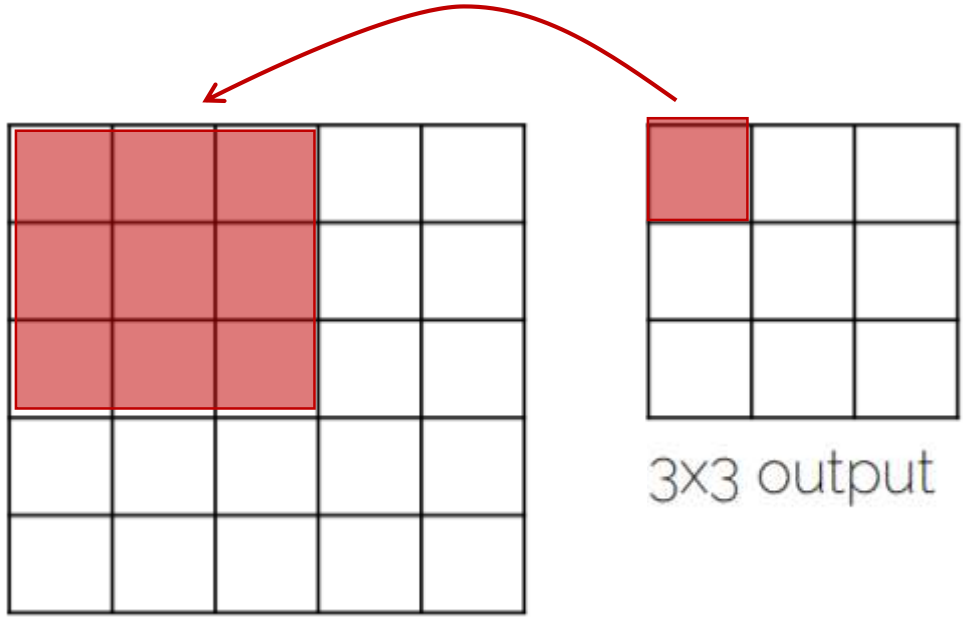


3x3 receptive field = 1 output pixel is connected to 9 input pixels

Source: Niessner and Leal-Taixé, Introduction to Deep Learning

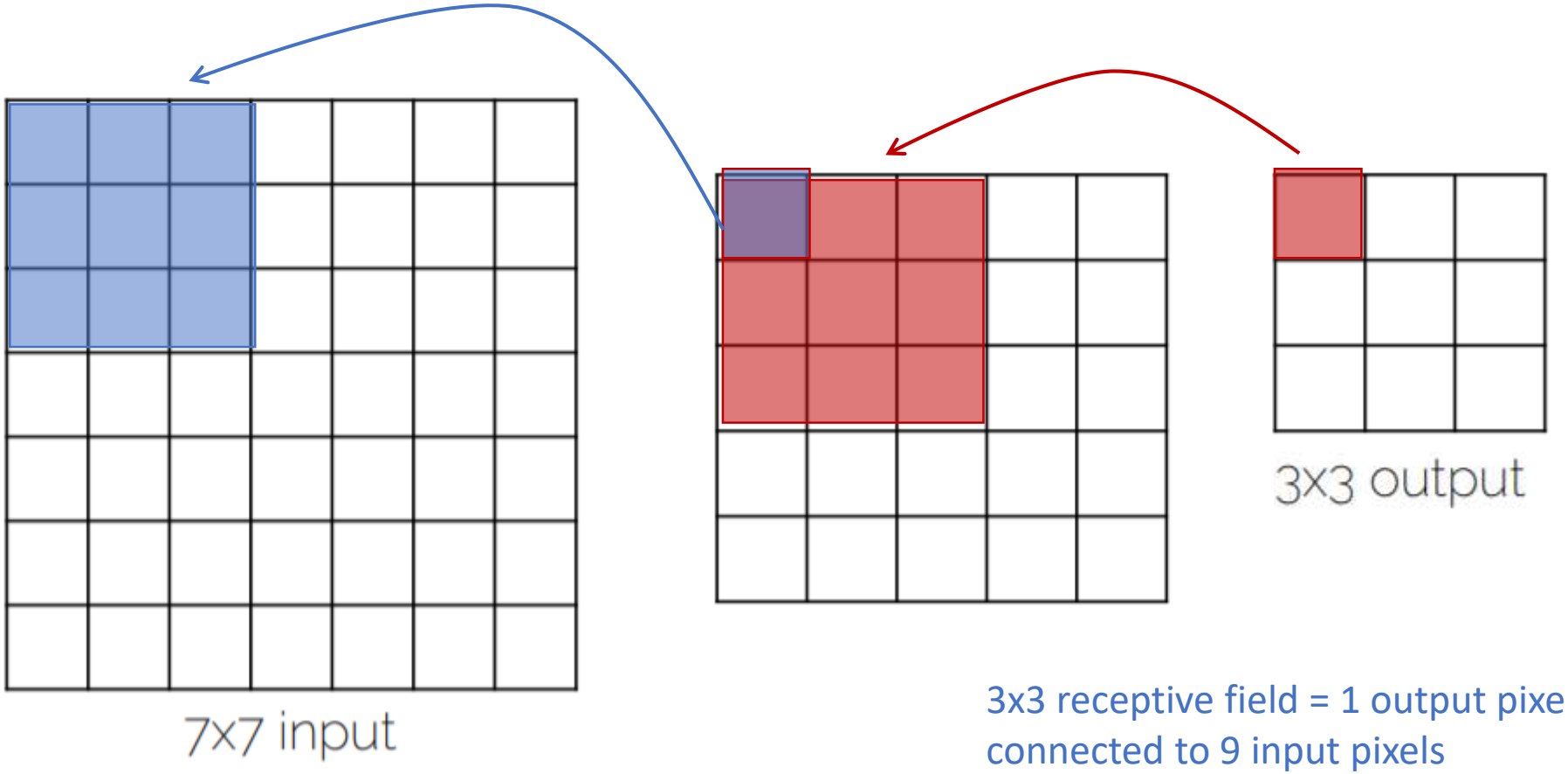
Receptive Field





3x3 receptive field = 1 output pixel is connected to 9 input pixels

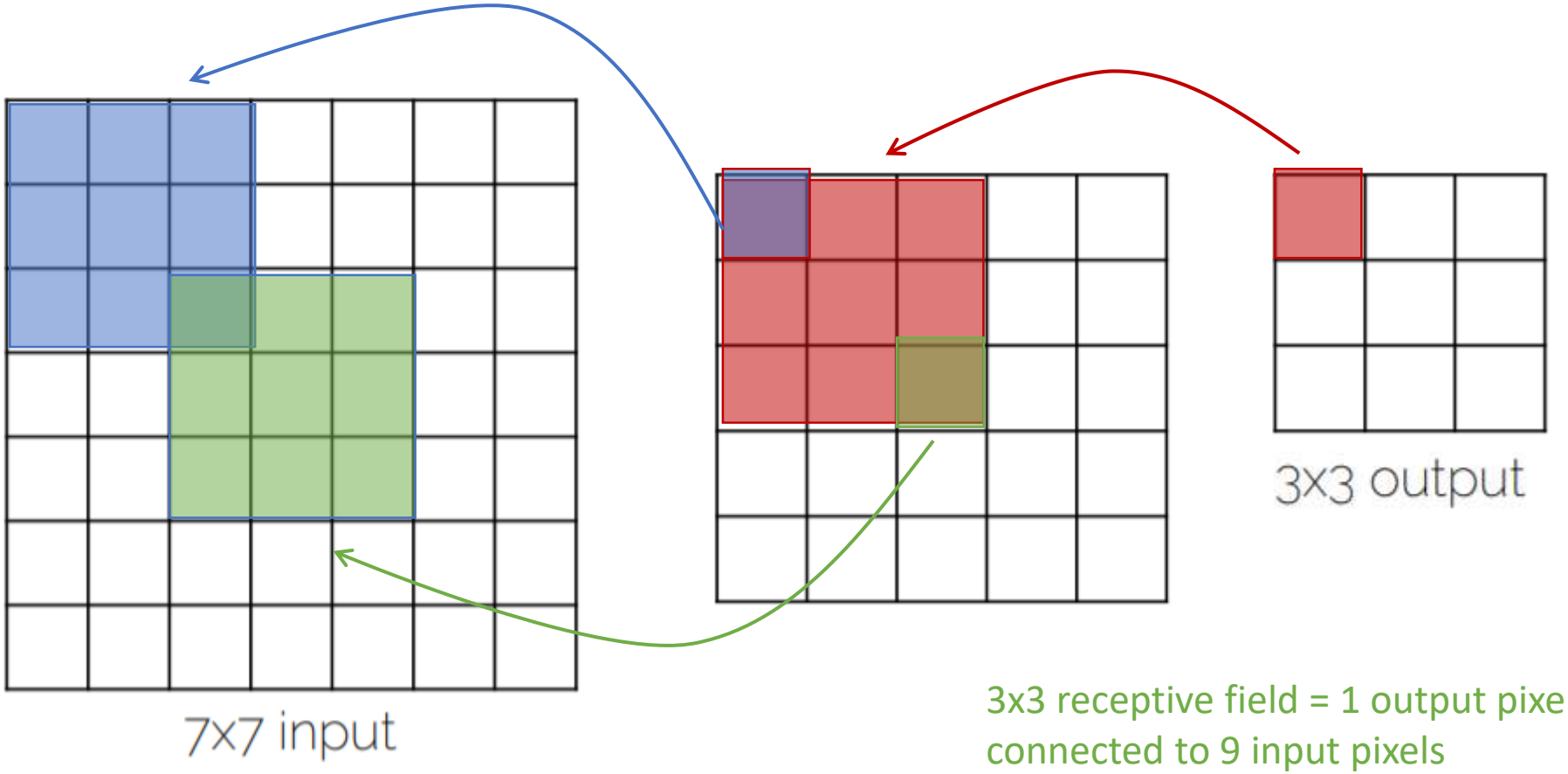
Receptive Field



3x3 receptive field = 1 output pixel is connected to 9 input pixels

Source: Niessner and Leal-Taixé, Introduction to Deep Learning

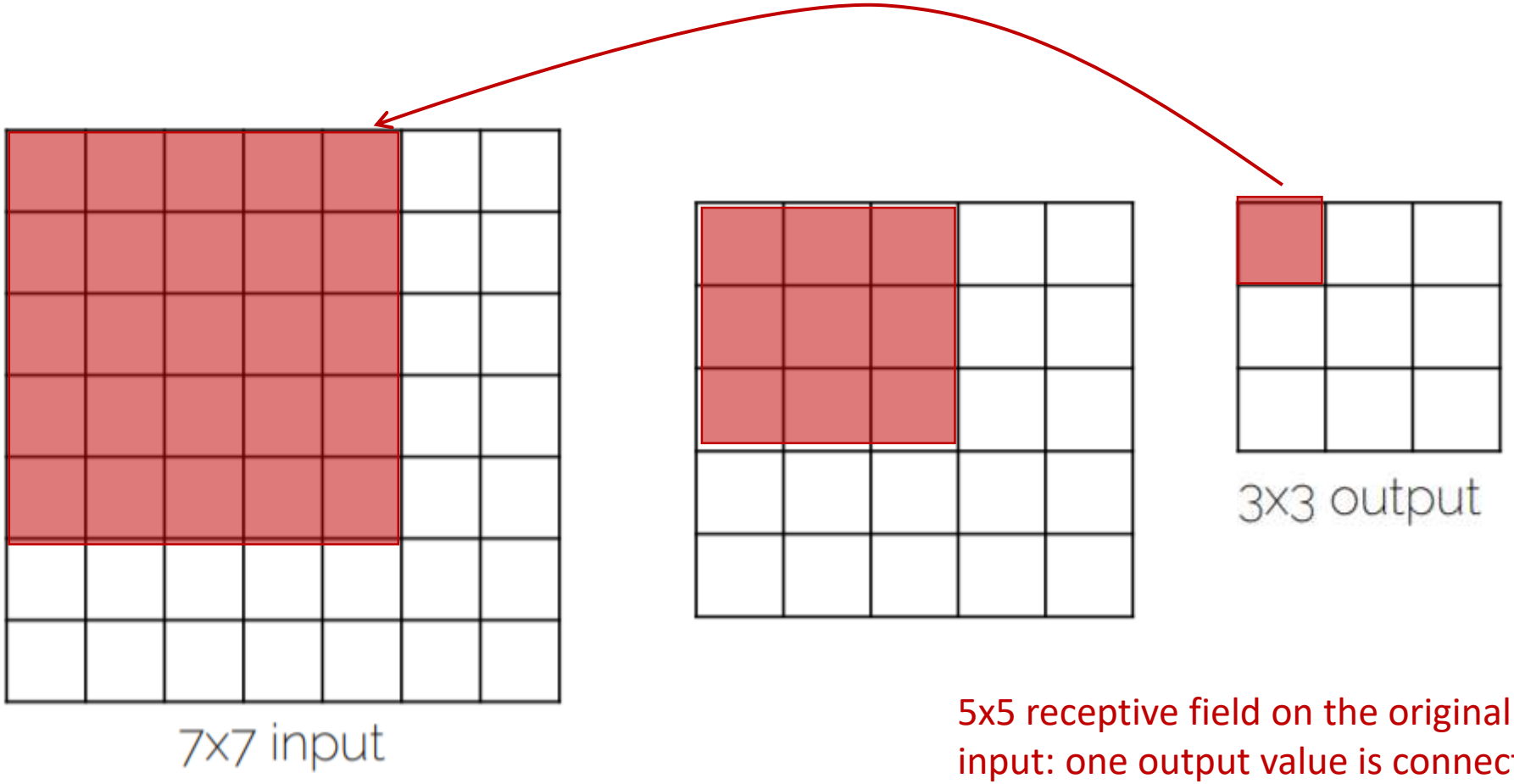
Receptive Field



3x3 receptive field = 1 output pixel is connected to 9 input pixels

Source: Niessner and Leal-Taixé, Introduction to Deep Learning

Receptive Field



5x5 receptive field on the original input: one output value is connected to 25 input pixels

Source: Niessner and Leal-Taixé, Introduction to Deep Learning